



Samueli
School of Engineering

CS-M151B

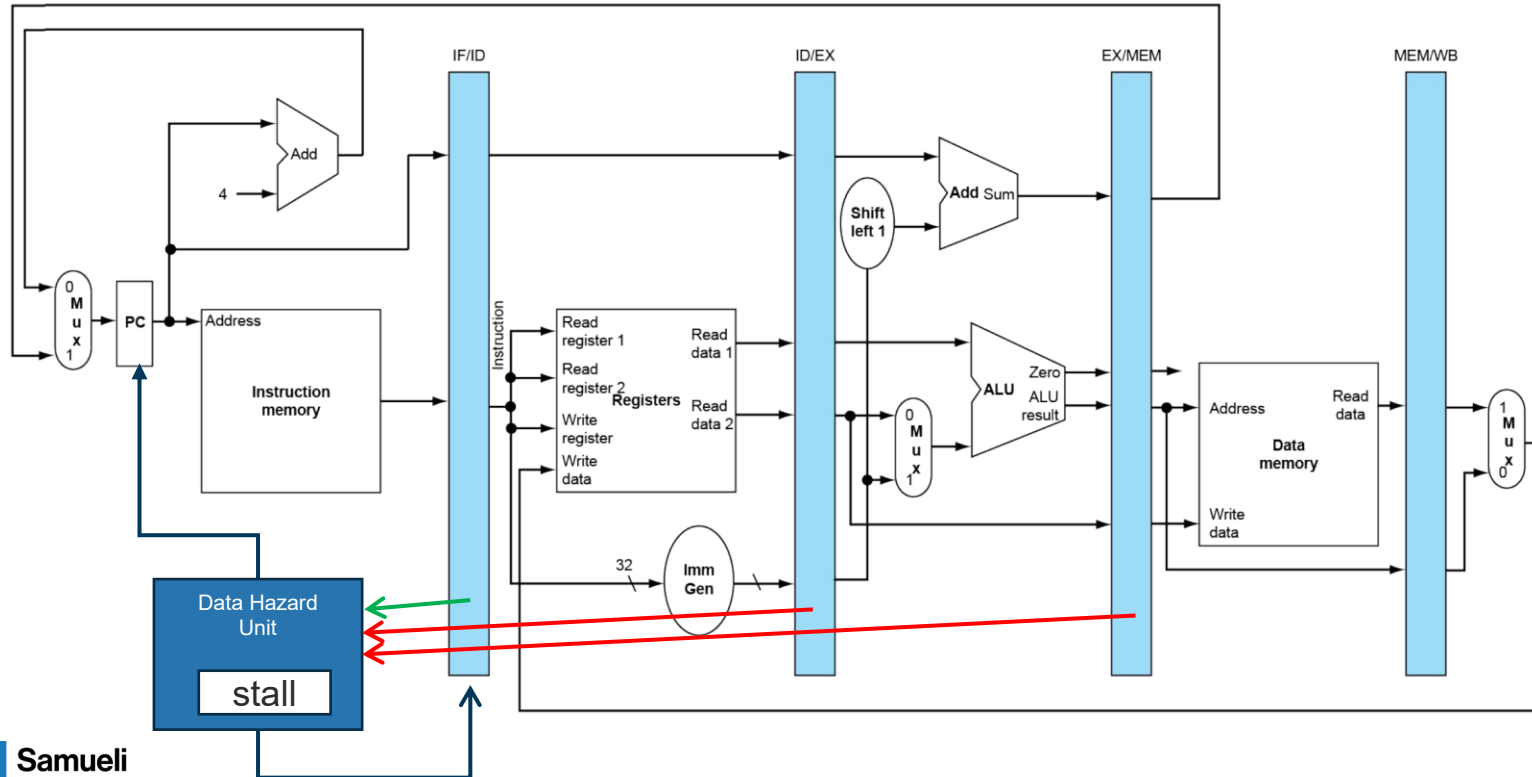
Computer Systems Architecture

Blaise Tine
UCLA Computer Science

Recap: Data Hazards

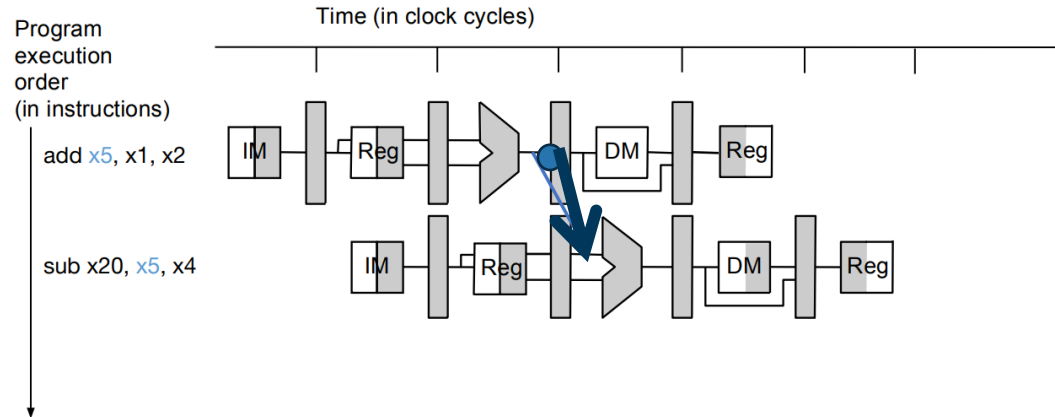
- Types of data Hazards
 - Read after write (RAW)
 - Write after write (WAW)
 - Write after read (WAR)
 - Read after read (RAR)

Recap: RAW Hazard Implementation



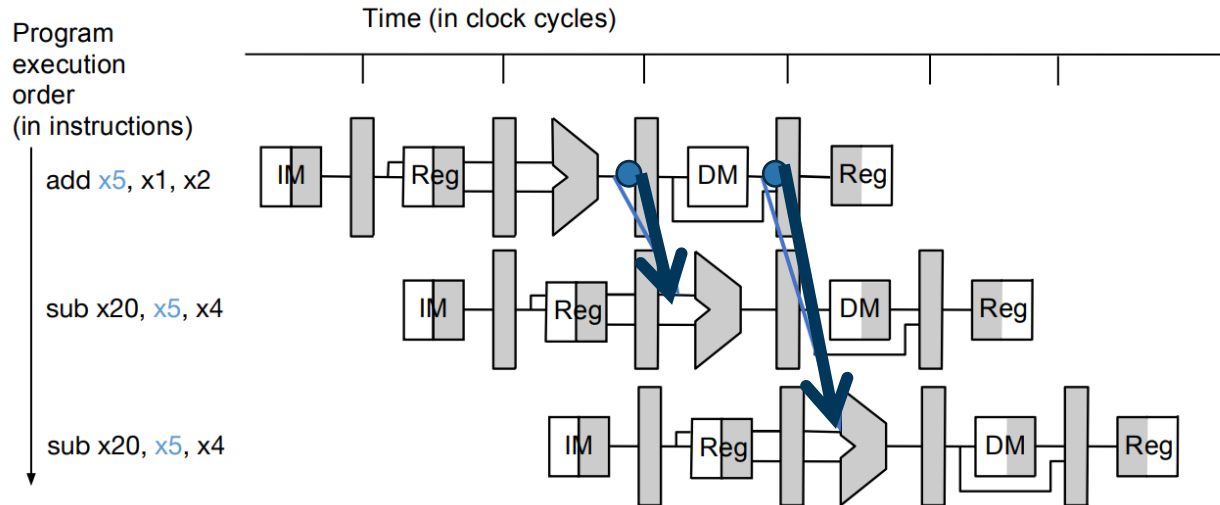
Recap: Resolving RAW hazards

- Data forwarding:
 - Redirect computed result directly to ALU
 - From EX/MEM to EX stage



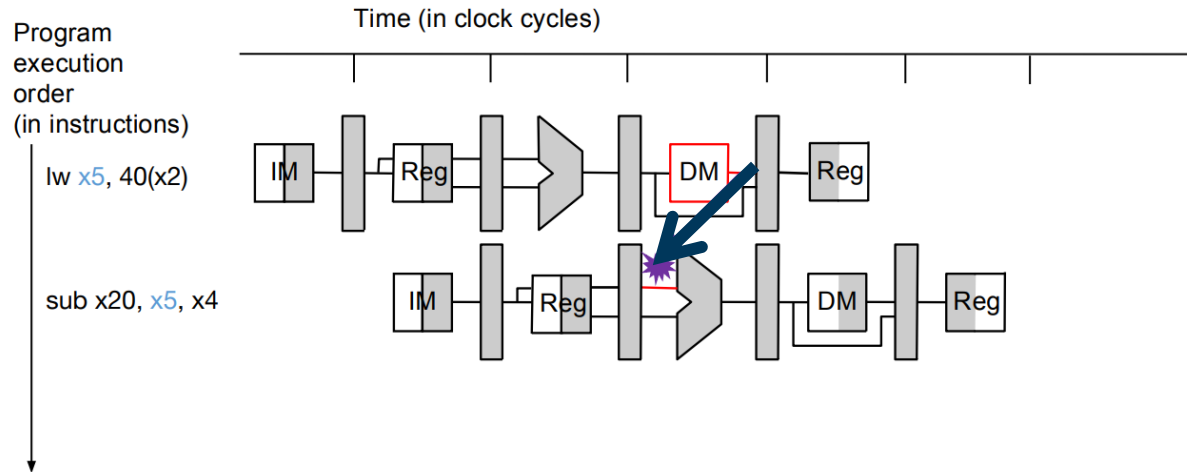
Recap: Resolving RAW hazards

- Data forwarding:
 - Long forwarding path
 - From MEM/WB to EX stage



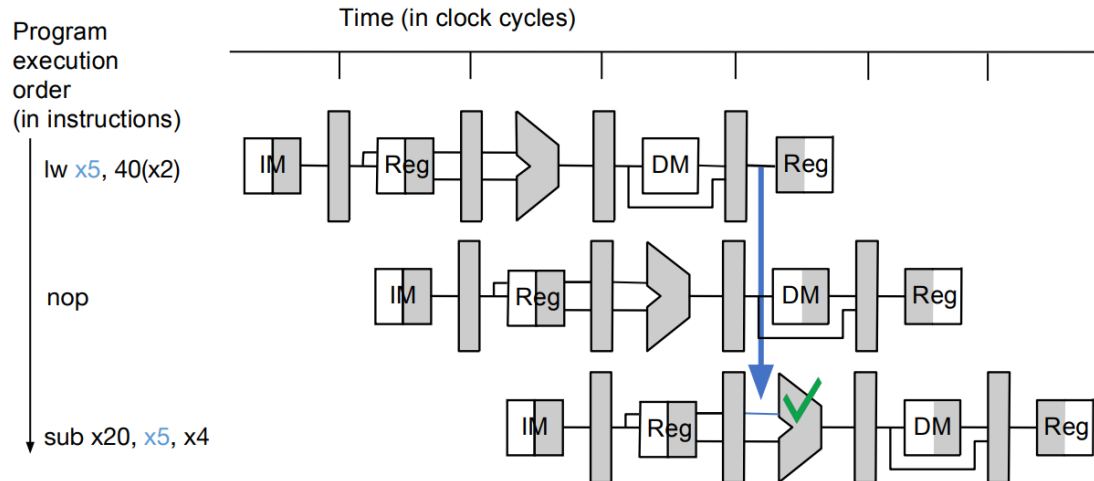
Recap: Resolving RAW hazards

- Data forwarding: Memory-ALU
 - Cannot do data forwarding from WB latch to EX stage
 - Not practical – not yet computed!



Recap: Resolving RAW hazards

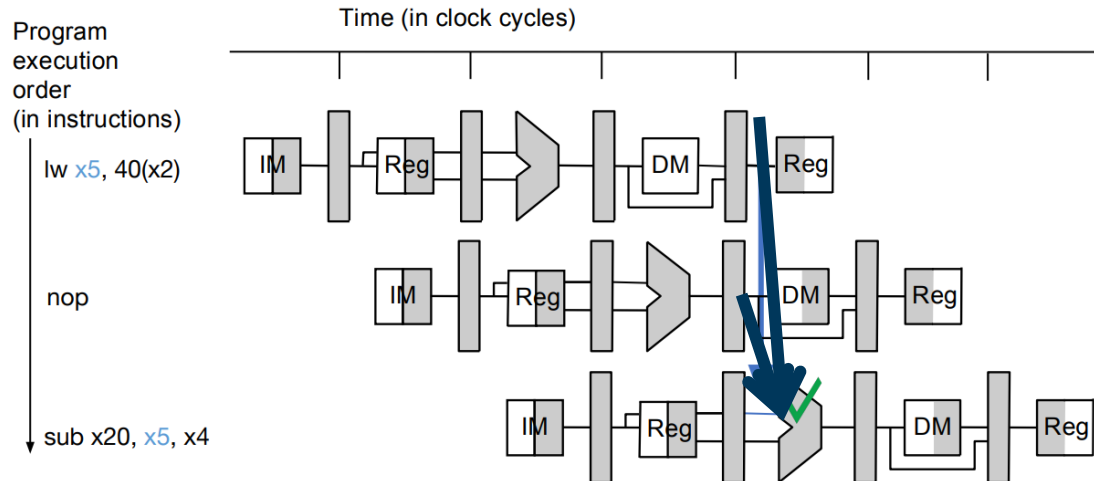
- Data forwarding: Memory-ALU
 - Add one bubble/NOP
 - Then forwarding from MEM/WB to EX stage



Resolving RAW hazards

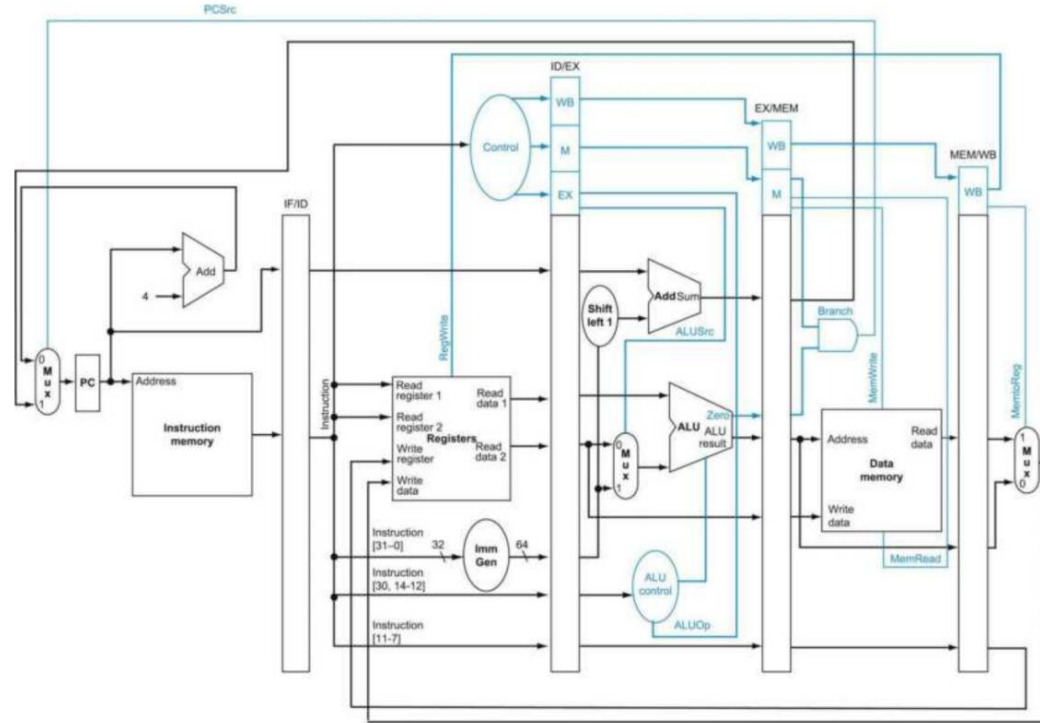
- Double Hazard Prioritization

- I0: add x1, x1, x2 (Produces value A)
- I1: add x1, x1, x3 (Produces value B)
- I2: add x1, x1, x4 (Needs to forward which value?)



Exercise: Data Forwarding Equations

- EX/MEM to EX forwarding
- MEM/WB to EX forwarding
- Load-Use Stall Detection

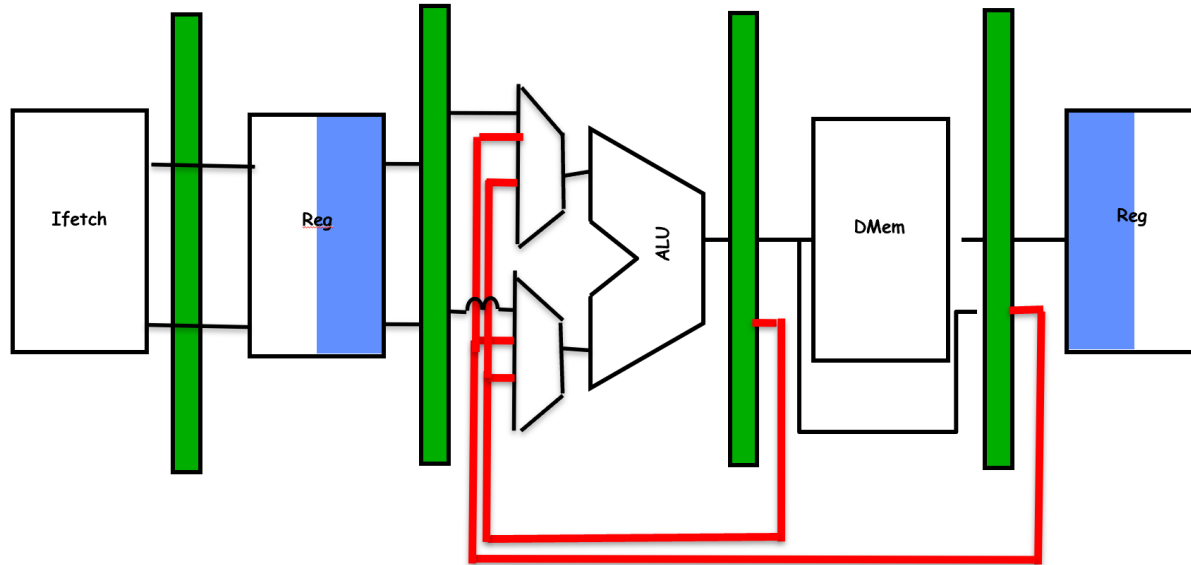


Resolving RAW hazards

- Data forwarding: Cost
 - Need extra wiring
 - Need multiplex to choose ALU source
 - Propagation delay may increase

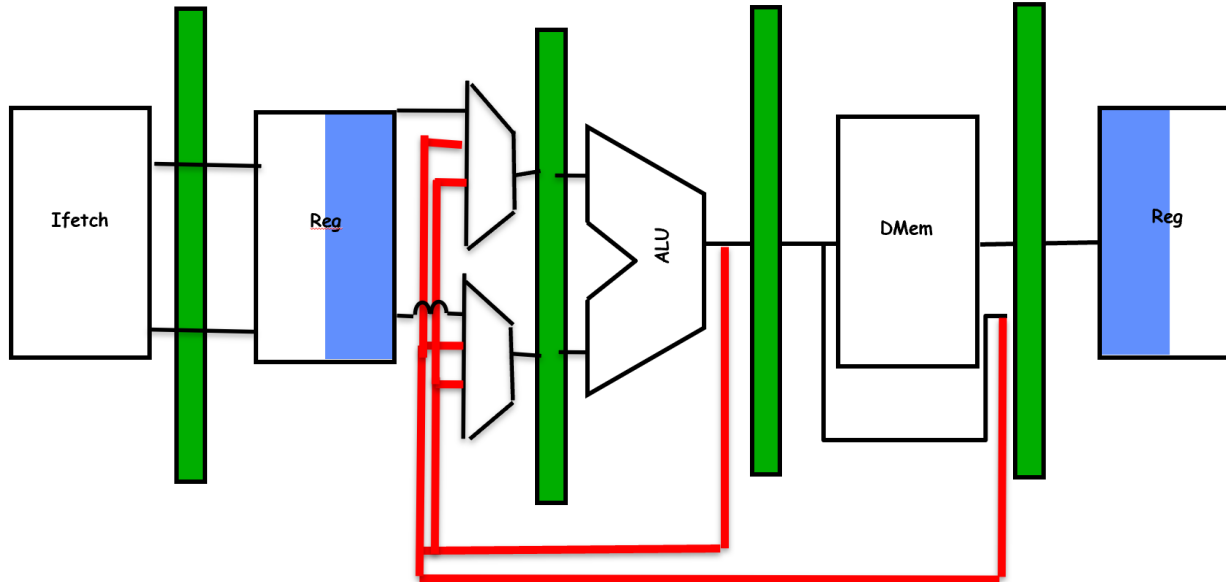
Resolving RAW hazards

- Data forwarding: design1
 - Insert MUXs before ALU



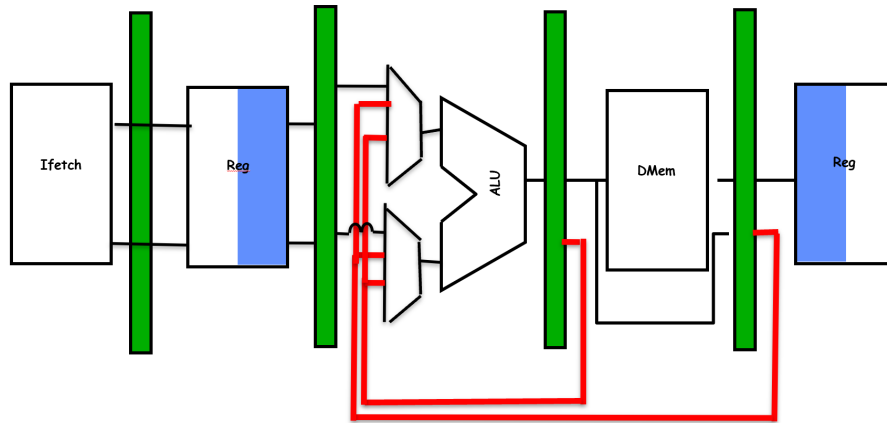
Resolving RAW hazards

- Data forwarding: design2
 - Insert MUXs before register

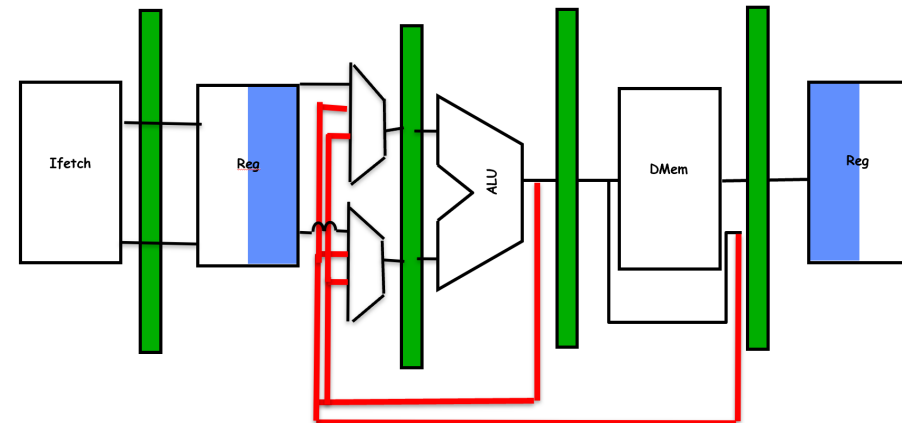


Resolving RAW hazards

- Data forwarding: which design is better?
 - The one with shorter propagation delay



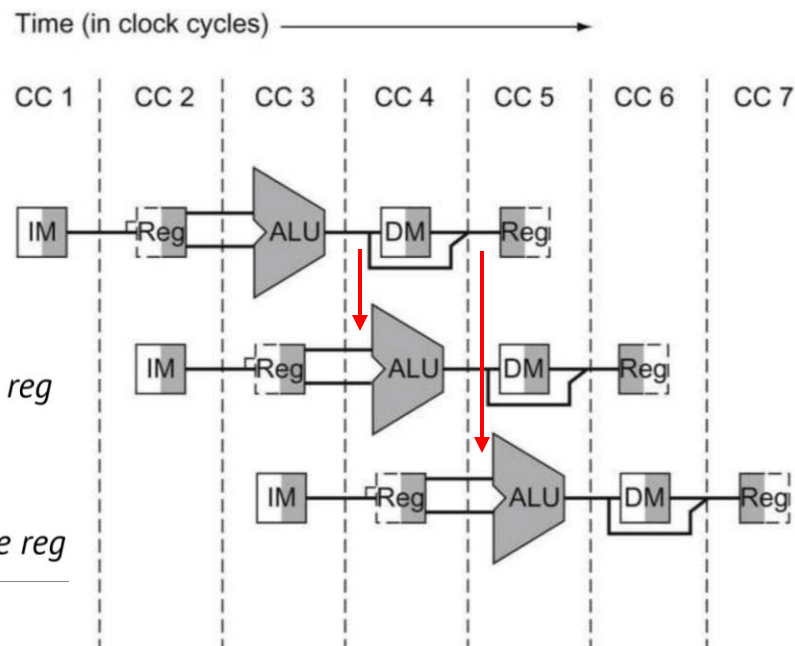
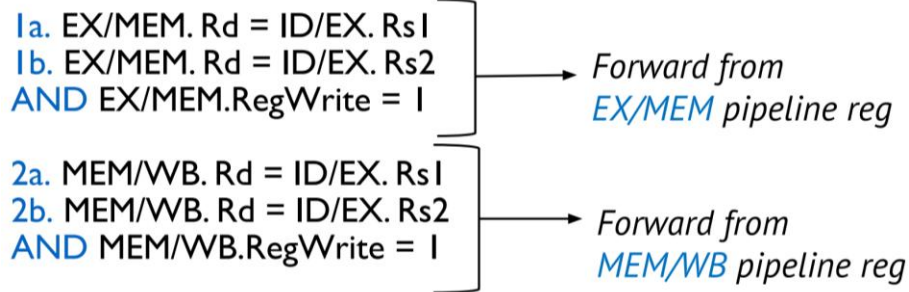
Design1



Design2

Resolving RAW hazards

- Data forwarding: detect RAW forwarding?
 - $(rd == x0) \Rightarrow$ no write



Resolving RAW hazards

- Software solutions:
 - Use instruction scheduling to reorder instructions to minimize hazards

```
1. ld x2, 0(x1) // Load: x2 = Memory[x1 + 0]
2. add x3, x2, x4 // Add: x3 = x2 + x4, but x2 is not ready
3. sub x5, x6, x7 // Subtract: x5 = x6 - x7 (independent of the above)
```



```
1. ld x2, 0(x1) // Load: x2 = Memory[x1 + 0]
2. sub x5, x6, x7 // Subtract: x5 = x6 - x7 (independent of the load)
3. add x3, x2, x4 // Add: x3 = x2 + x4, x2 is now ready
```

Control Hazards

- Understanding branches
 - e.g. for loop

```
int main() {  
    for (int i = 0; i < 10; i++) {  
        // loop body  
    }  
    return 0;  
}
```



```
main:  
    li t0, 0          # i = 0  
    li t1, 10         # n=10  
loop:  
    addi t0, t0, 1     # i = i + 1  
    bne t0, t1, loop   # if (i != n) => loop  
exit:  
    li a0, 0          # return 0  
    ret
```

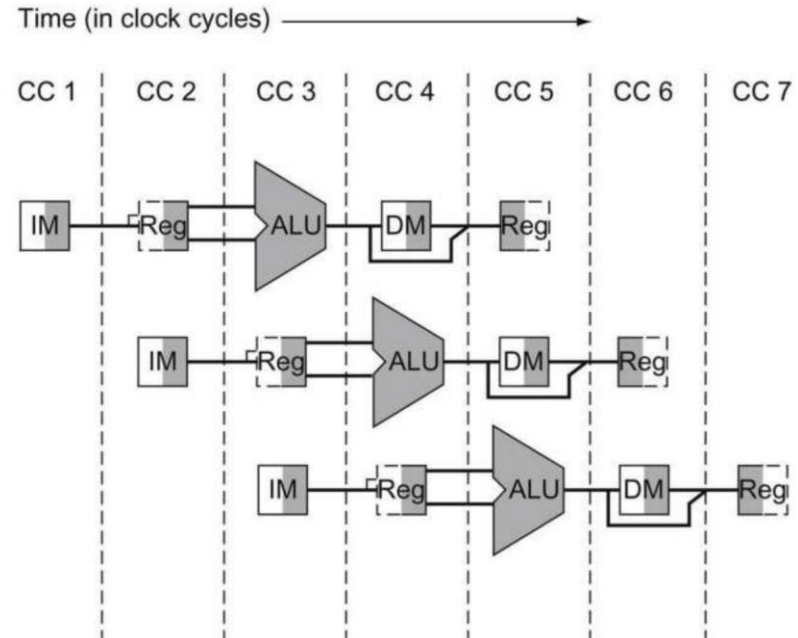

Control Hazards

- Data and control dependence
 - I0: addi x5, x5, 1
 - I1: bne x5, x6, loop
- Whether to branch?
 - (x5 == x6)
- Where to branch?
 - PC + imm

Program
execution
order
(in instructions)

I0

I1



Control Hazards

RISC-V branches

- Conditional branches
 - BEQ r1, r2, imm; // if (r1 == r2) => PC = PC + imm
- Unconditional branches
 - JAL rd, imm; // rd = PC + 4; PC = PC + imm
 - JALR rd, rb, imm; // rd = PC + 4; PC = rb + imm

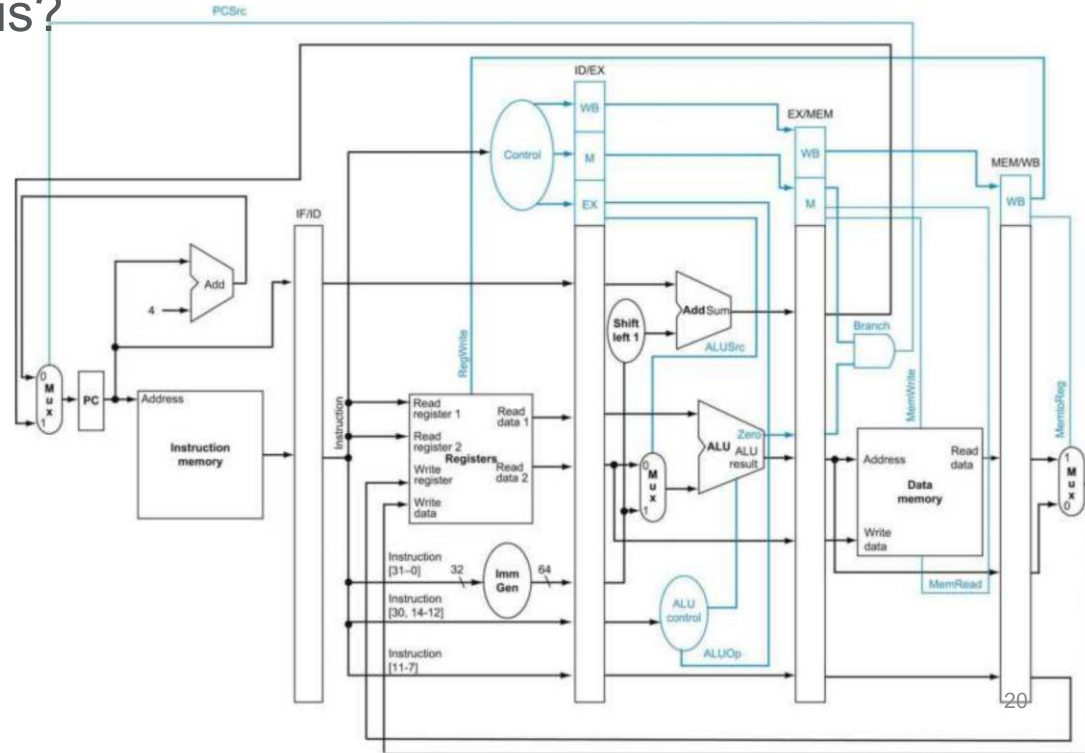
Control Hazards

CPU must answer 3 questions:

- Which type of branch?
- Whether to branch?
- Where to branch?

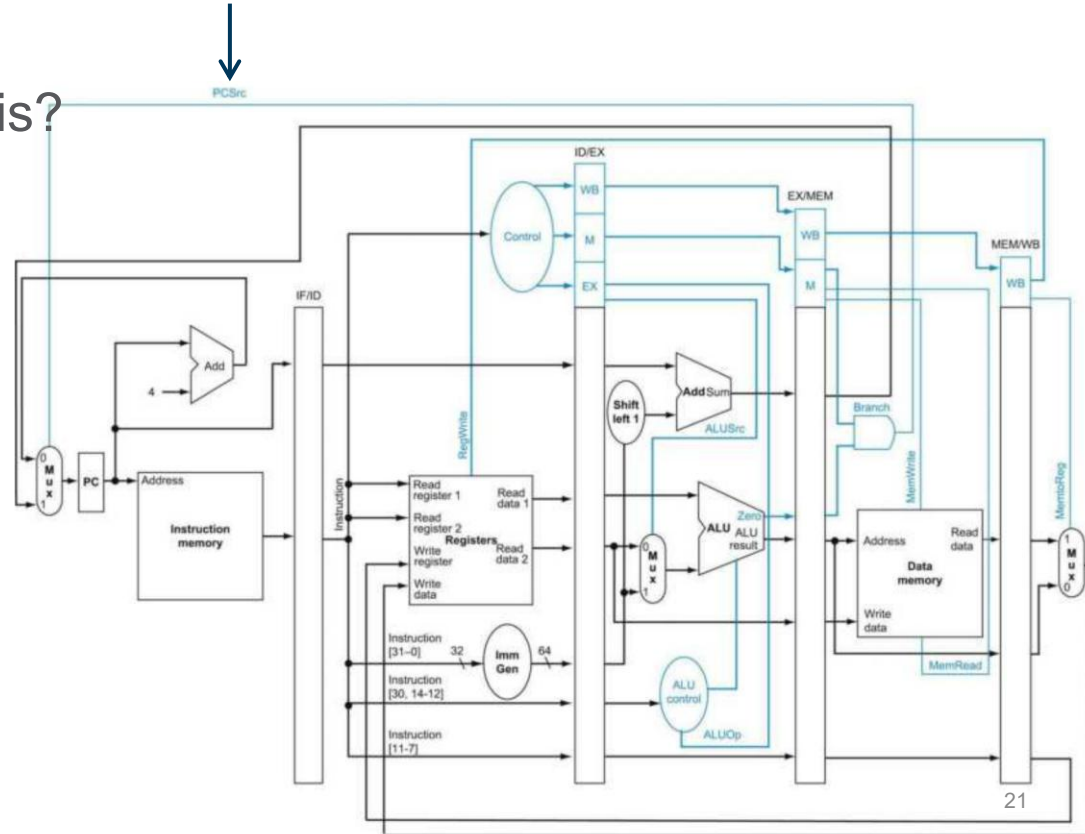
Control Hazards

- Which type of branch
 - Where to determine this?
 - ID stage



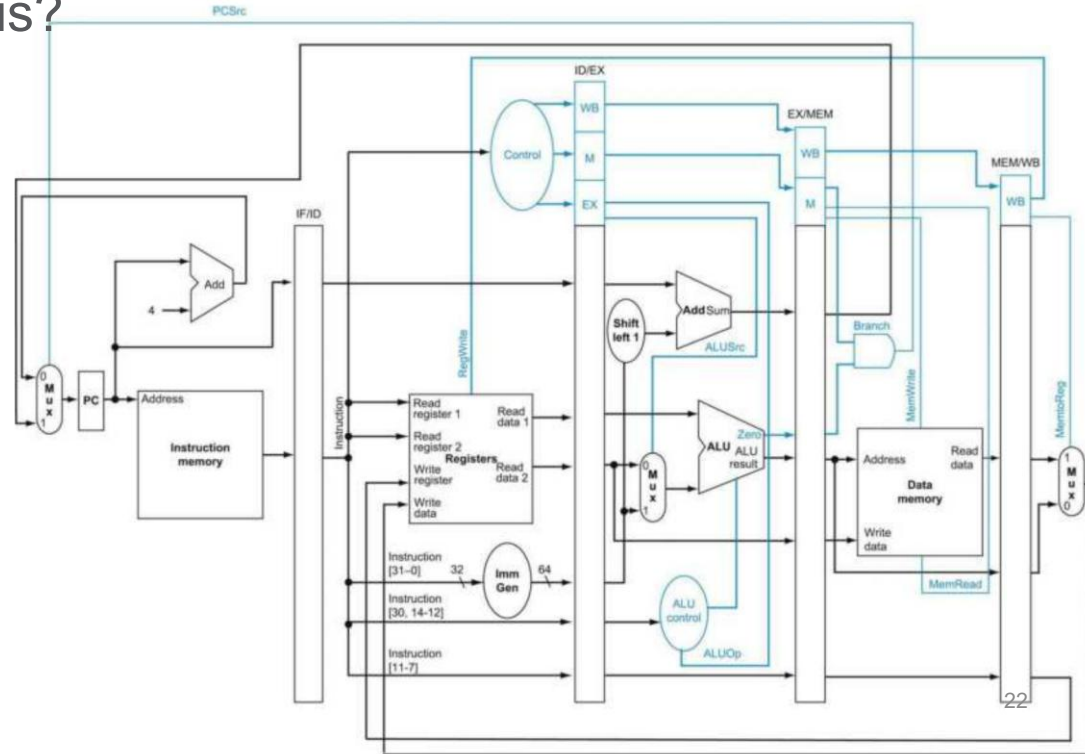
Control Hazards

- Whether to branch
 - Where to determine this?
 - MEM stage (PCSrc)



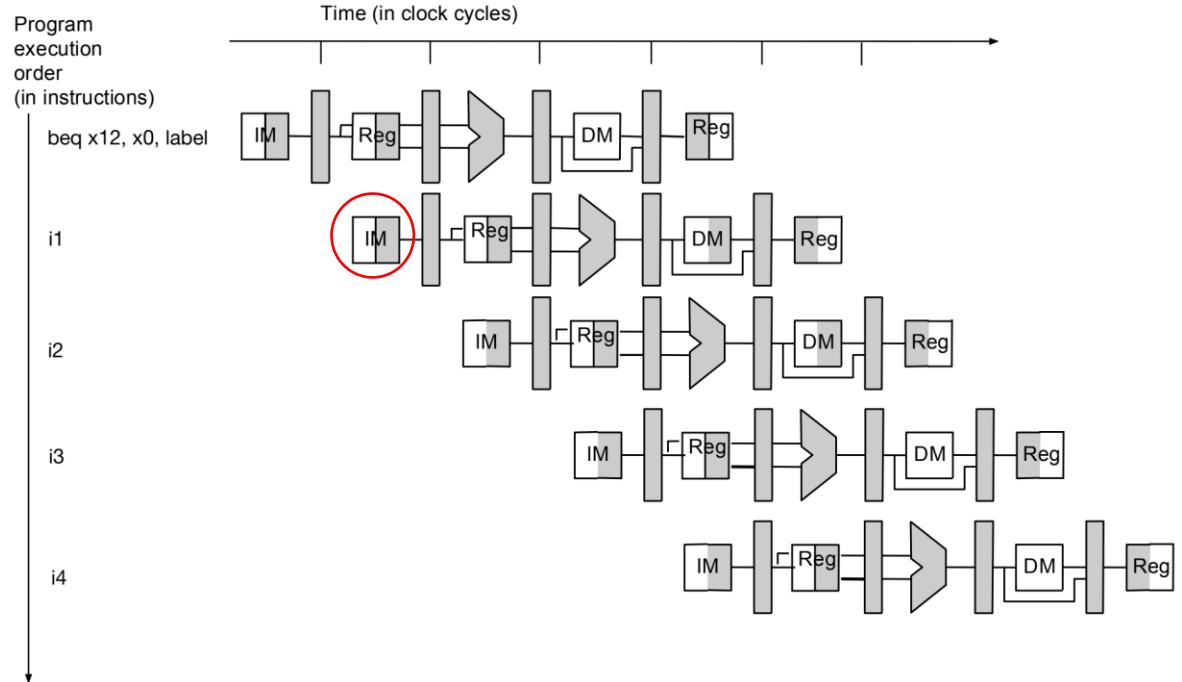
Control Hazards

- Where to branch
 - Where to determine this?
 - MEM stage



Control Hazards

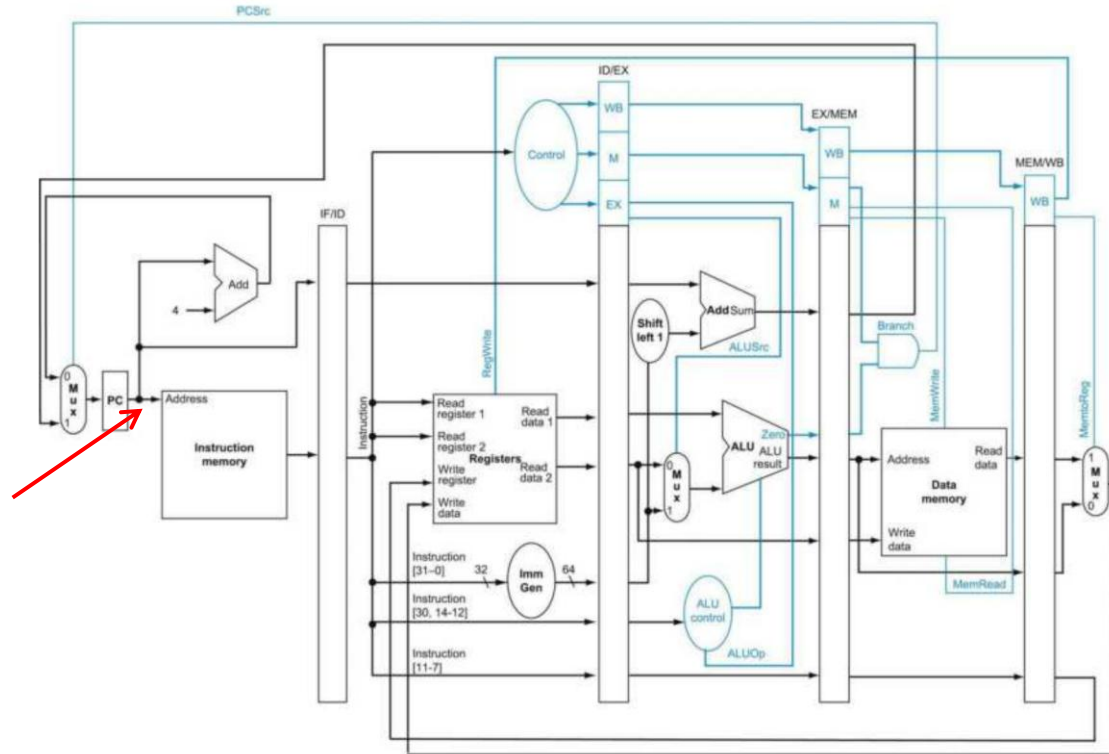
- Which stage should stall?
 - IF stage
- Why?
 - **Need new PC**



Control Hazards

- When is next PC available?
 - When PCSrc ready
 - When PC mux ready
 - MEM

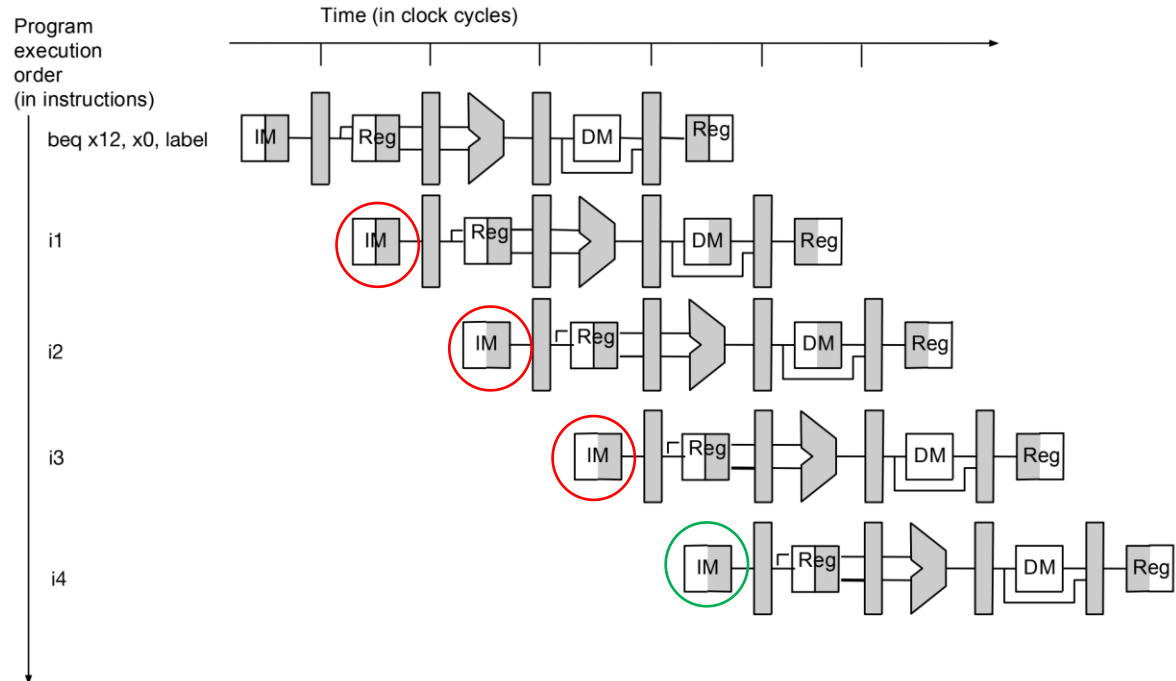
Assume IM access and PC update happen in same cycle, in IF stage



Control Hazards

- Stall cycles needed?
 - 3 cycles

Can we do better?



Control Hazards Handling

- Software Solution: Insert NOPs
 - S/W insert NOPs in delay slots
 - Pros:
 - No H/W cost
 - Cons:
 - Bad performance: still stalling
 - S/W depends on H/W stall count

Control Hazards Handling

- Software Solution: Delayed branch
 - S/W insert useful instructions in delay slots
 - Pros:
 - No H/W cost
 - Improved performance
 - Cons:
 - Hard to find useful instructions
 - S/W depends on H/W stall count

Control Hazards Handling

- Hardware Solution: Stall the pipeline
 - Pros:
 - S/W independence
 - Minimal H/W cost
 - Cons:
 - Bad performance

Control Hazards Handling

- Hardware Solution: Speculation
 - Pros:
 - S/W independence
 - Better performance
 - Cons:
 - H/W cost

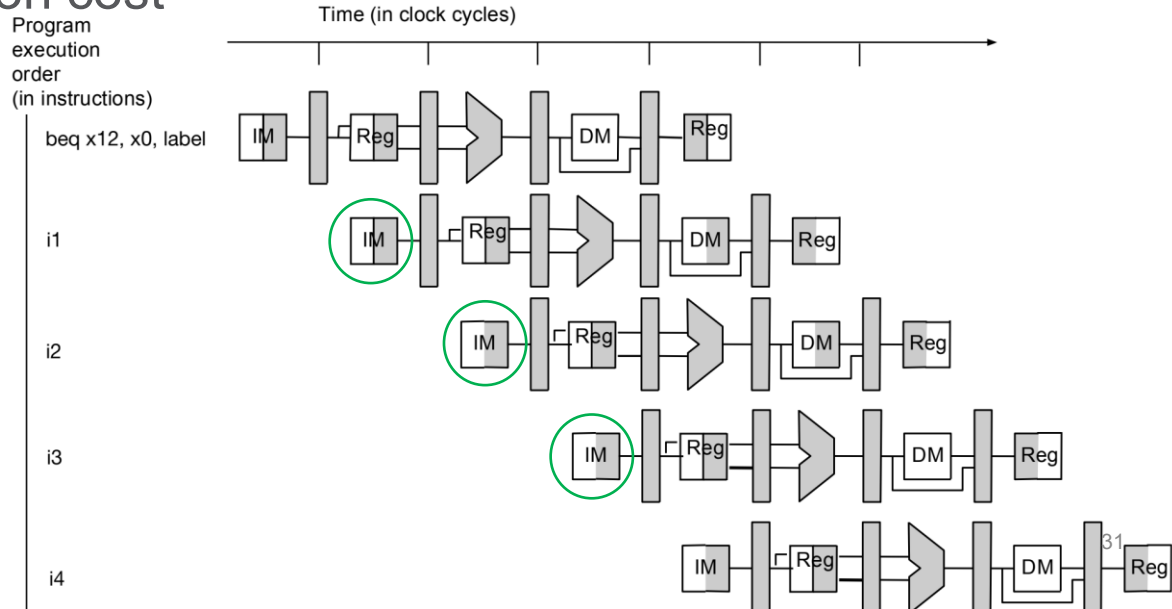
Attendance

- Please Fill the form with today's keyword to register your attendance.



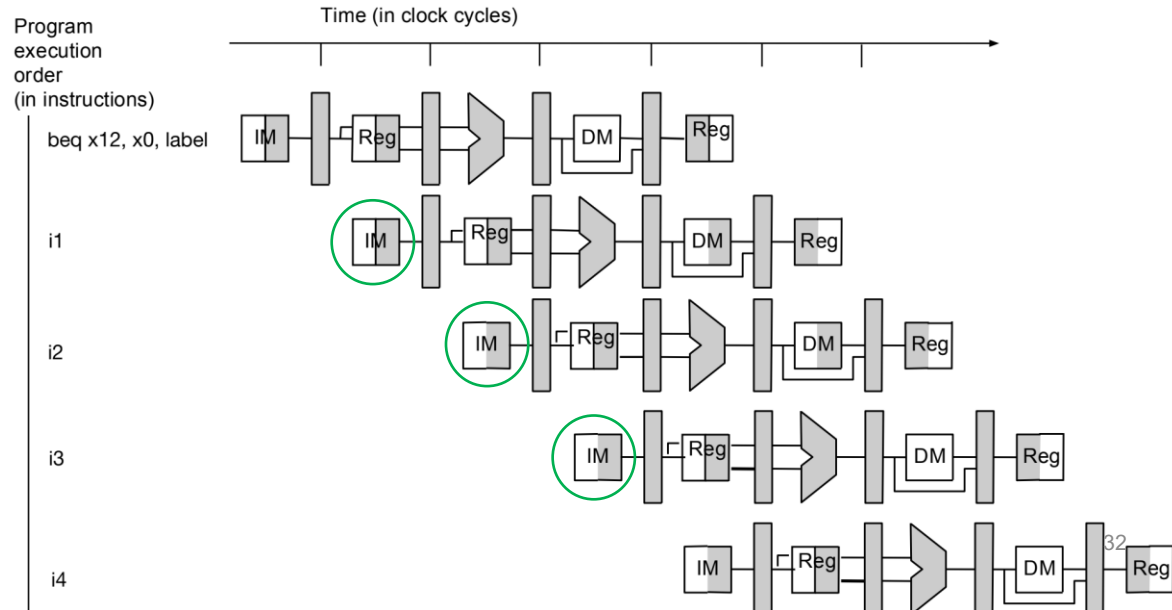
Control Hazards: Speculation

- Predict: Not taken
 - Emulate delay branch
 - Pros: no wasted cycles
 - Cons: mis-prediction cost



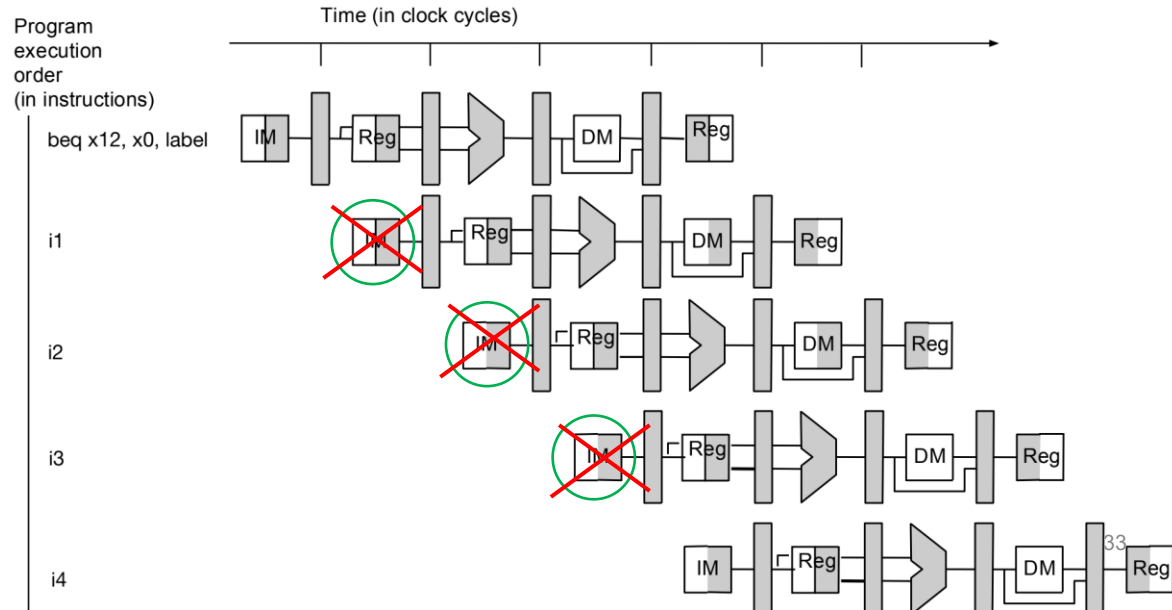
Control Hazards: Speculation

- Predict: Not taken
 - How to detect misprediction?
 - Branch condition is taken
 - When?
 - MEM stage



Control Hazards: Speculation

- Predict: Not taken
 - How to recover from misprediction?
 - Flush the pipeline (invalidate each previous stage)
 - Reset PC



Control Hazards: Speculation

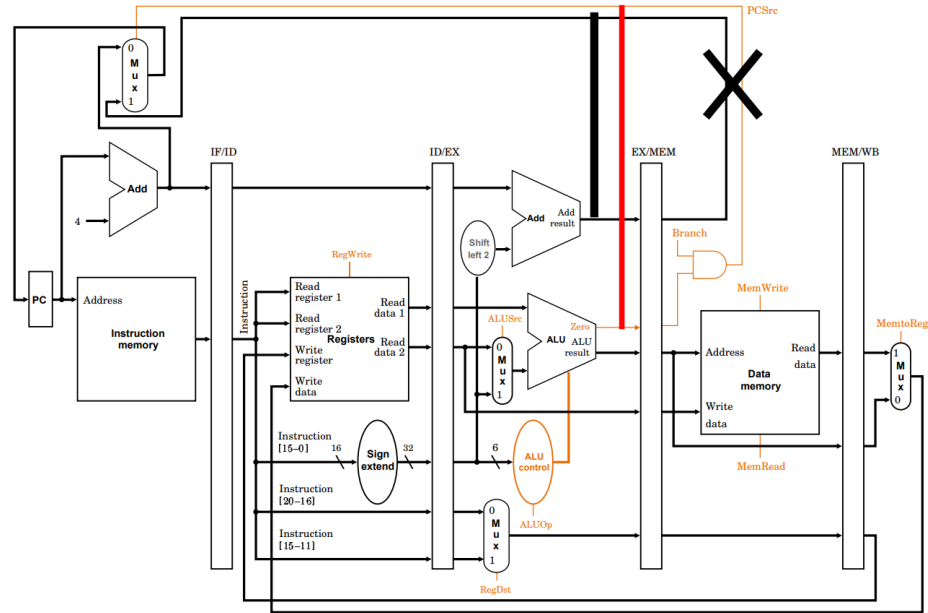
- Practical Branch stats
 - About 20% of instructions is control-flow
 - 50% of forward branches (if-then-else) are taken
 - 90% of backward branches (loops) are taken
 - 30% of all branches are not-taken

Control Hazards: Speculation

- Practical Branch stats
 - About 20% of instructions is control-flow
 - 50% of forward branches (if-then-else) are taken
 - 90% of backward branches (loops) are taken
 - 30% of all branches are not-taken
- Not-taken prediction accuracy?
 - Only 30%
 - Cost: 3 cycles

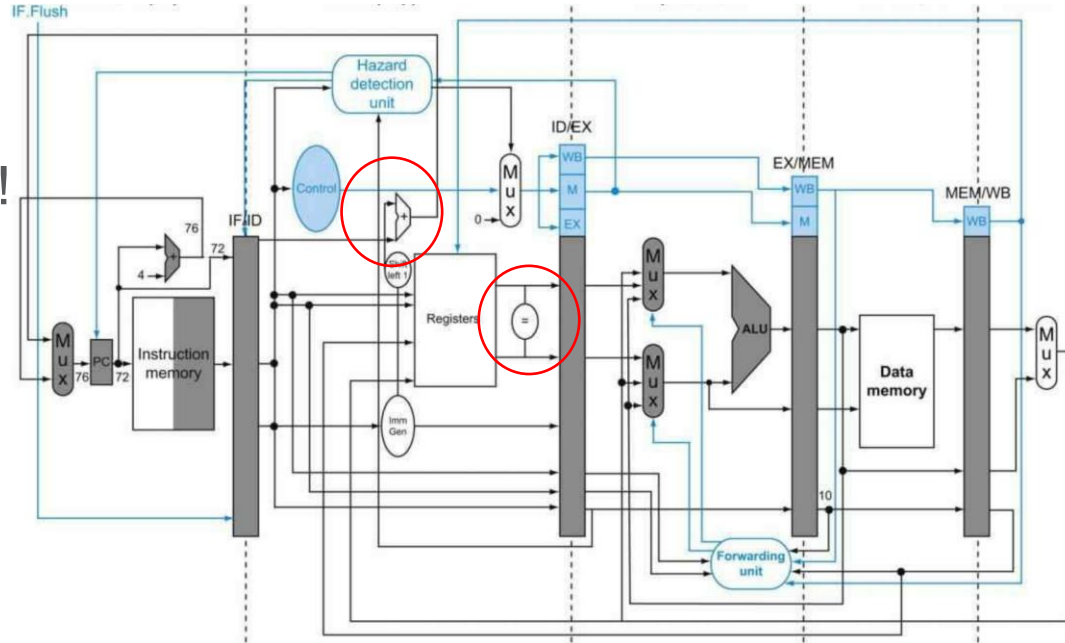
Control Hazards: Speculation

- Reducing Misprediction Cost
 - Resolving branches sooner
 - Making it 2 cycles can be done



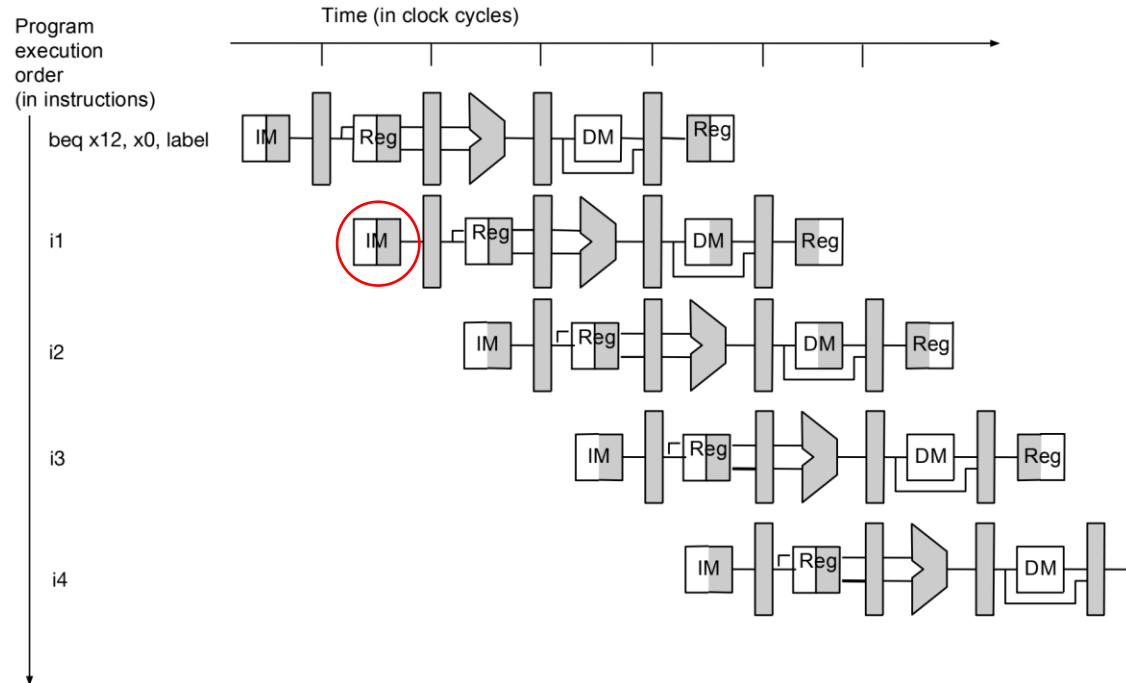
Control Hazards: Speculation

- Reducing Misprediction Cost
 - More aggressive solution: **Unconditional Branch only**
 - Move NextPC to DE
 - Move PC + imm to DE
- Warning: r1/r2 RAW
 - Need data forwarding!



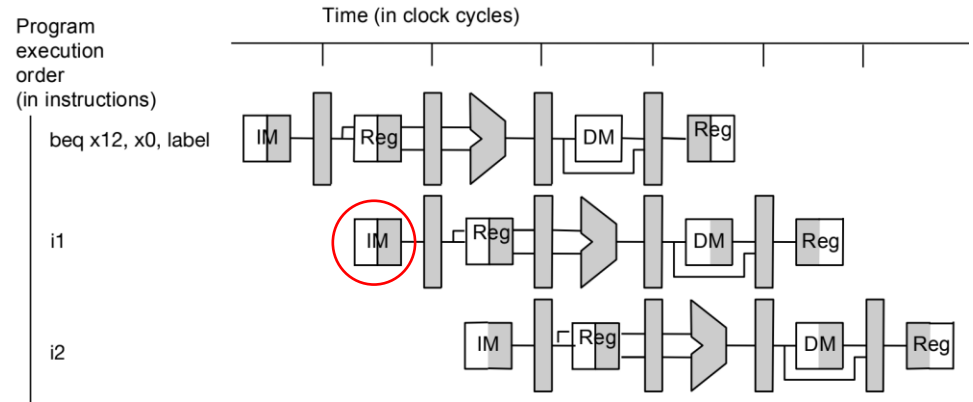
Control Hazards: Speculation

- Predict: Always Taken
- What do we need?
 - nextPC
 - At fetch stage



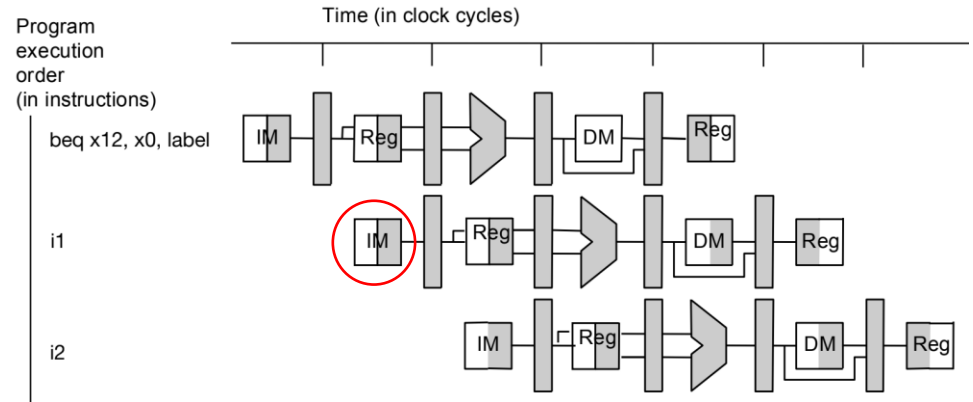
Control Hazards: Speculation

- Predict: Always Taken
 - How to obtain nextPC?
 - We can predict



Control Hazards: Speculation

- Predict: Always Taken
 - How to obtain nextPC?
 - We can predict



Control Hazards

RISC-V branches

- Conditional branches
 - `BEQ r1, r2, imm;    // if (r1 == r2) => PC = PC + imm`
- Unconditional branches
 - `JAL rd, imm;        // rd = PC + 4; PC = PC + imm`
 - `JALR rd, rb, imm;   // rd = PC + 4; PC = rb + imm`

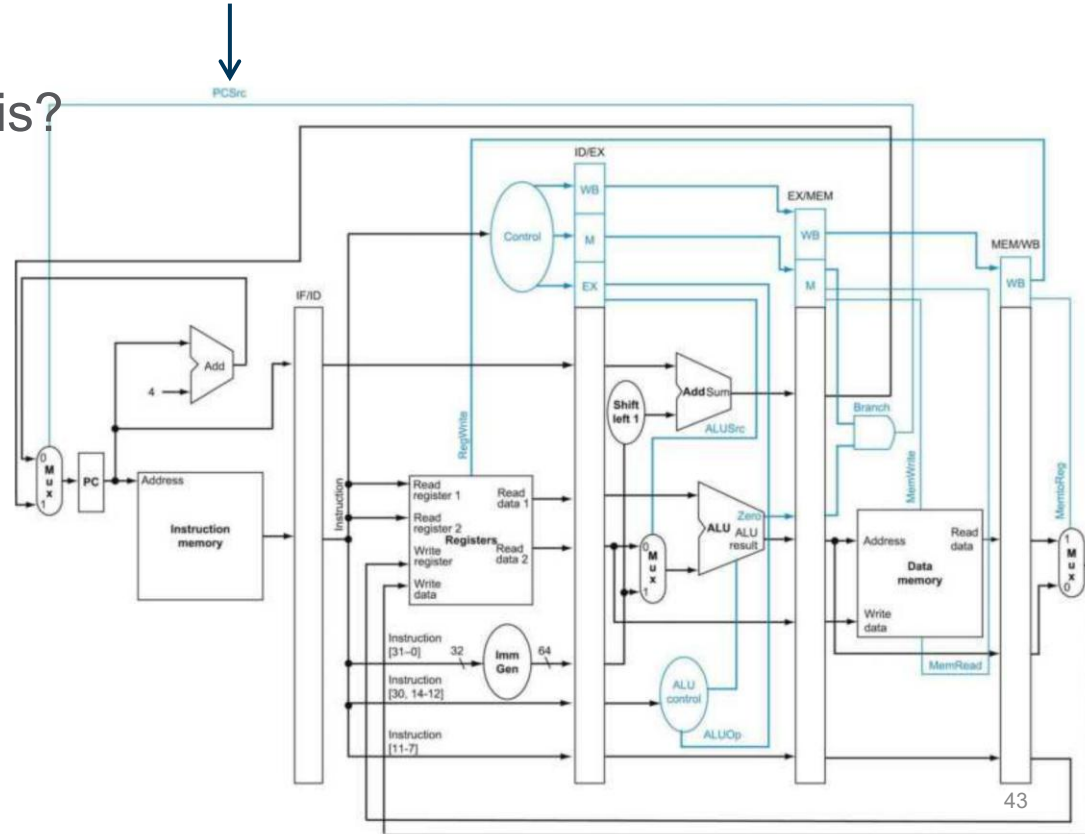
Control Hazards

CPU must answer 3 questions:

- Which type of branch? e.g. BEQ / JAL
- Whether to branch? e.g. (rs1 == rs2)
- Where to branch? e.g. PC + imm

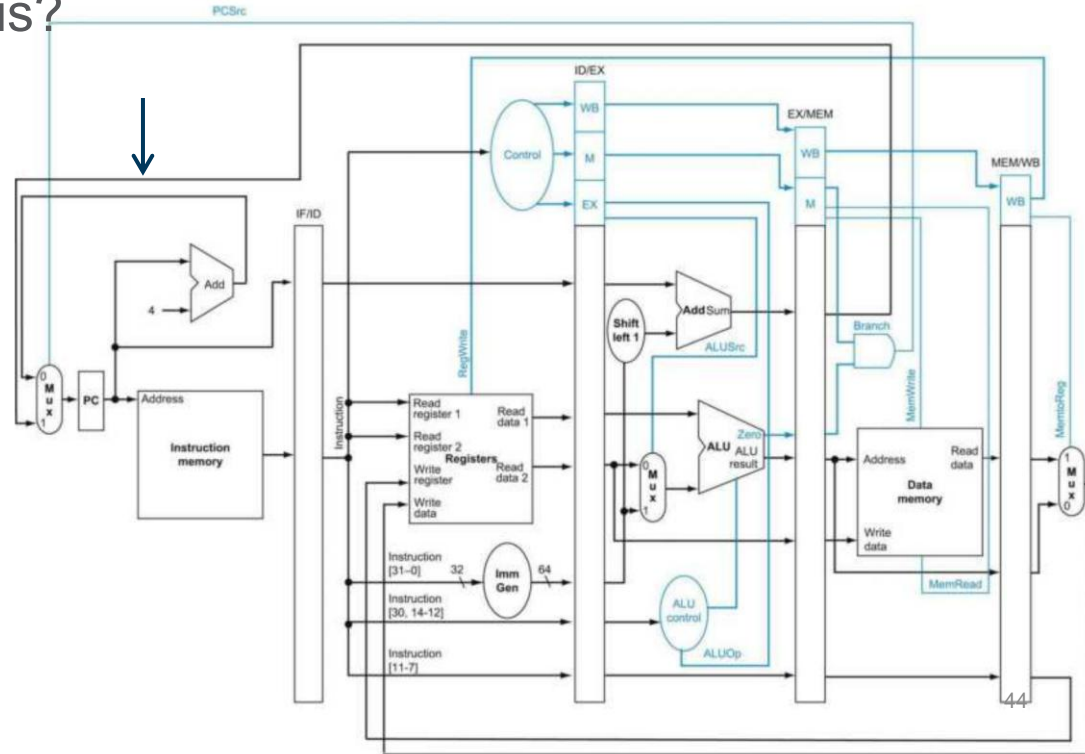
Control Hazards

- Whether to branch
 - Where to determine this?
 - MEM stage (PCSrc)



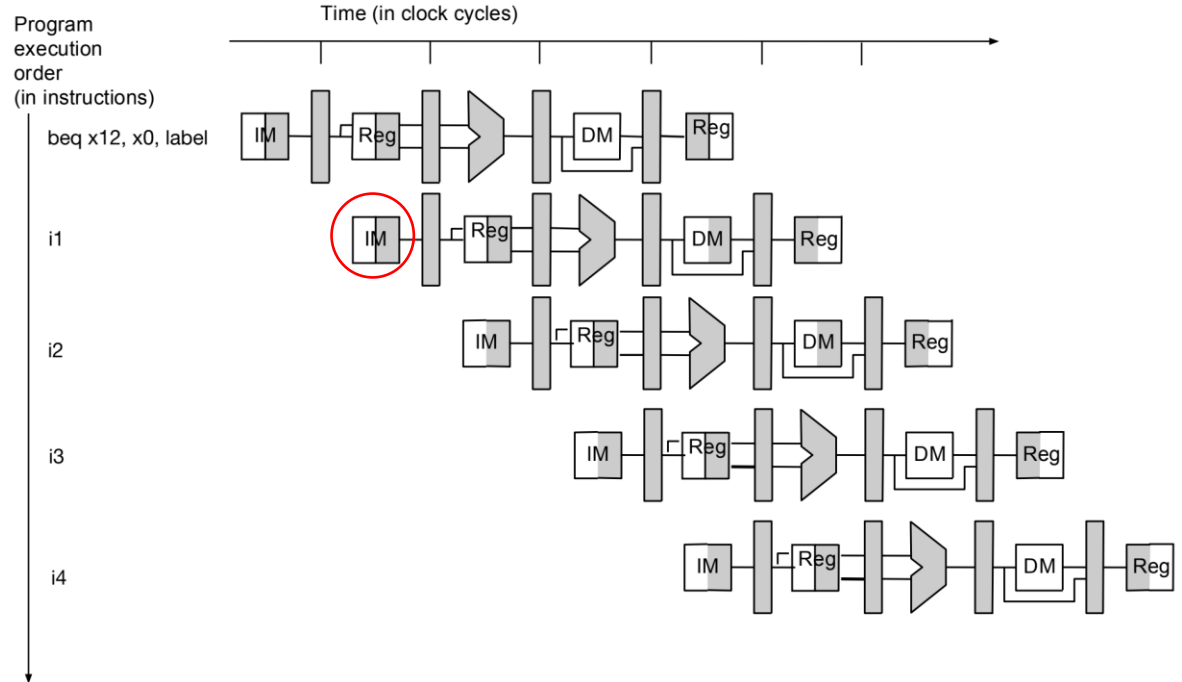
Control Hazards

- Where to branch
 - Where to determine this?
 - MEM stage



Control Hazards

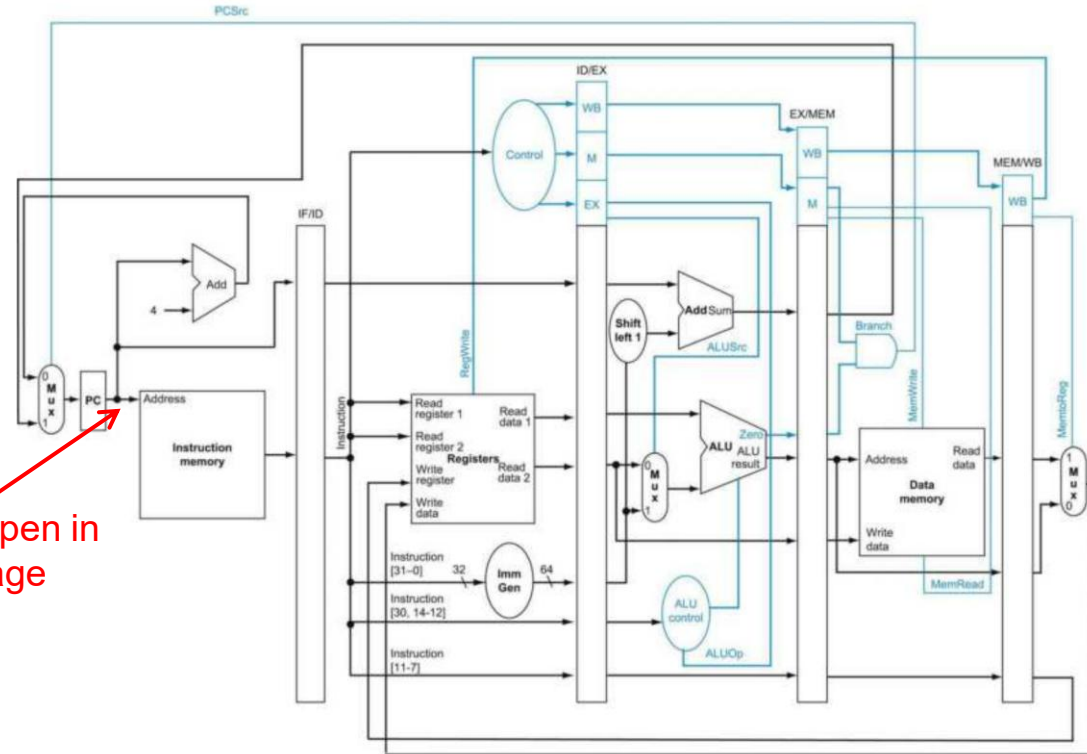
- Which stage should stall?
 - IF stage
- Why?
 - **Need new PC**



Control Hazards

- When is next PC available?
 - When PCSrc ready
 - MEM

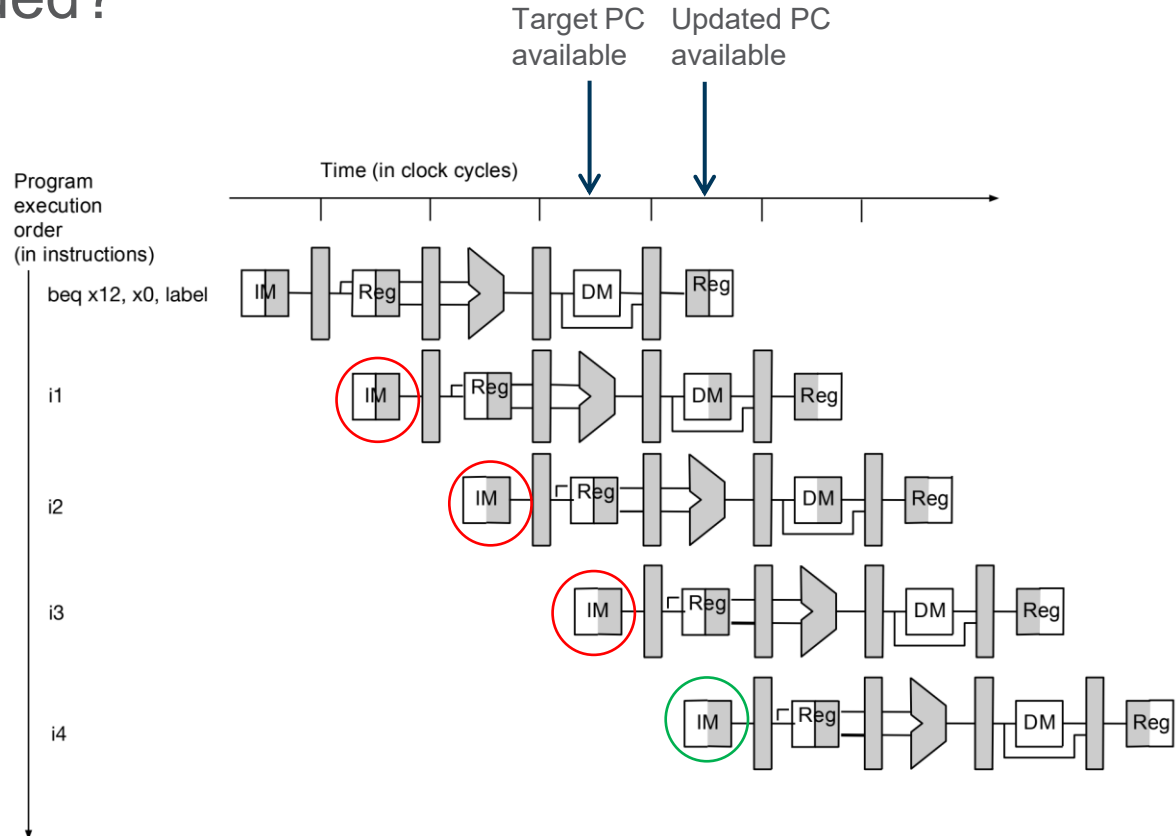
PC and IM read happen in same cycle, in IF stage



Control Hazards

- Stall cycles needed?
 - 3 cycles

Can we do better?



Control Hazards Handling

- Software Solution: Insert NOPs
 - S/W insert NOPs in delay slots
 - Pros:
 - No H/W cost
 - Cons:
 - Bad performance: still stalling
 - S/W depends on H/W stall count

Control Hazards Handling

- Software Solution: Delayed branch
 - S/W insert useful instructions in delay slots
 - Pros:
 - No H/W cost
 - Improved performance
 - Cons:
 - Hard to find useful instructions
 - S/W depends on H/W stall count

Control Hazards Handling

- Hardware Solution: Stall the pipeline
 - Pros:
 - S/W independence
 - Minimal H/W cost
 - Cons:
 - Bad performance

Control Hazards Handling

- Hardware Solution: Speculation
 - Pros:
 - S/W independence
 - Better performance
 - Cons:
 - H/W cost

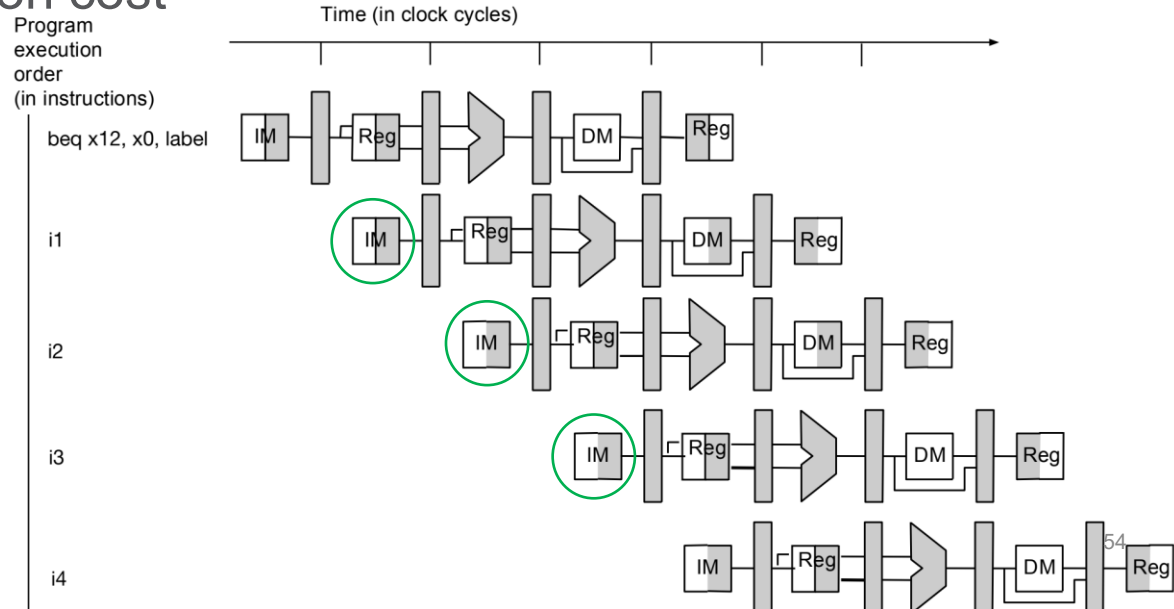
Control Hazards: Speculation

- Practical Branch stats
 - About 20% of instructions is control-flow
 - 50% of forward branches (if-then-else) are taken
 - 90% of backward branches (loops) are taken
 - 30% of all branches are not-taken
 - Empirical studies (e.g. SPEC benchmarks)
- Not-taken prediction accuracy?
 - Only 30%
 - Cost: 3 cycles

Whiteboard: Backward branches

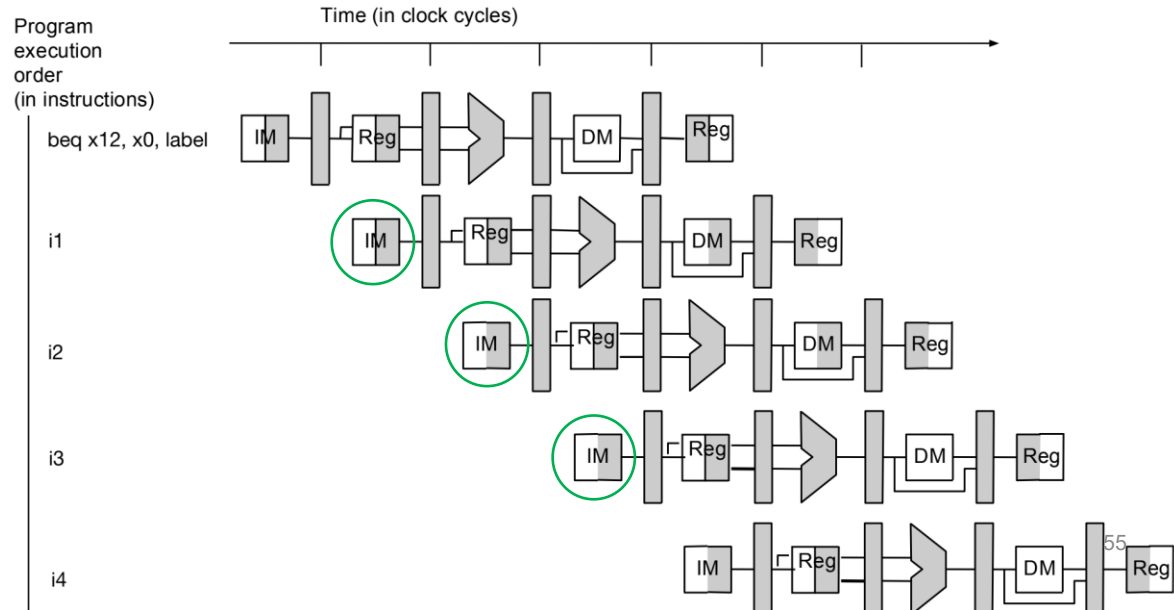
Control Hazards: Speculation

- Predict: Not taken
 - Emulate delay branch
 - Pros: no wasted cycles
 - Cons: mis-prediction cost



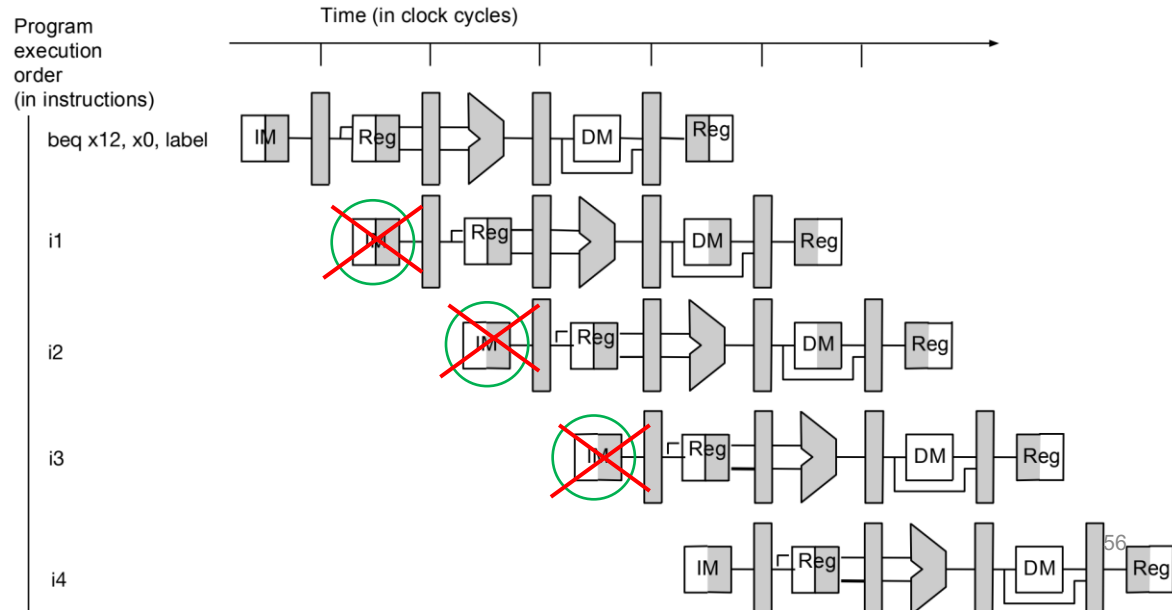
Control Hazards: Speculation

- Predict: Not taken
 - How to detect misprediction?
 - Branch condition is taken
 - When?
 - MEM stage



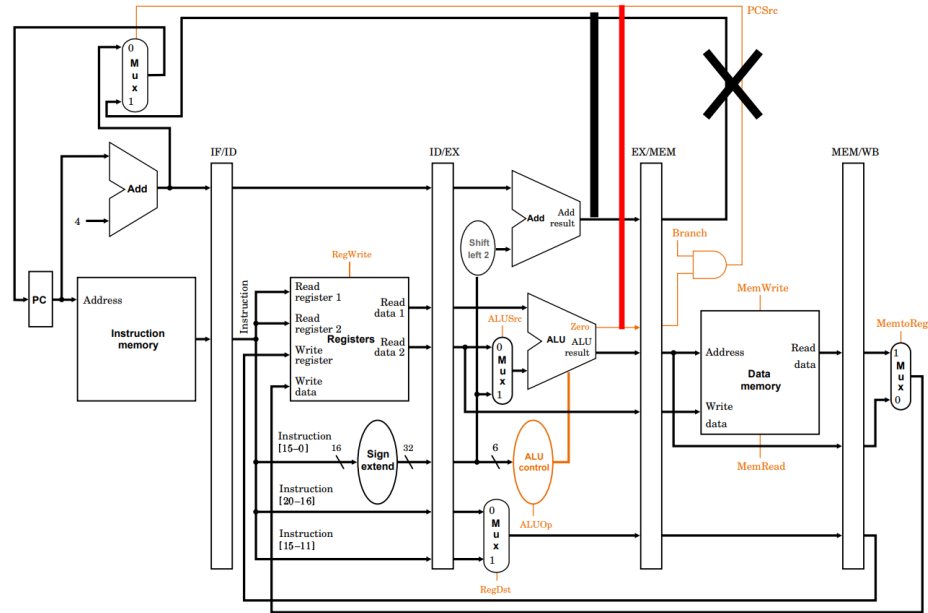
Control Hazards: Speculation

- Predict: Not taken
 - How to recover from misprediction?
 - Flush the pipeline (invalidate previous stages IF, ID, EX)
 - Update PC



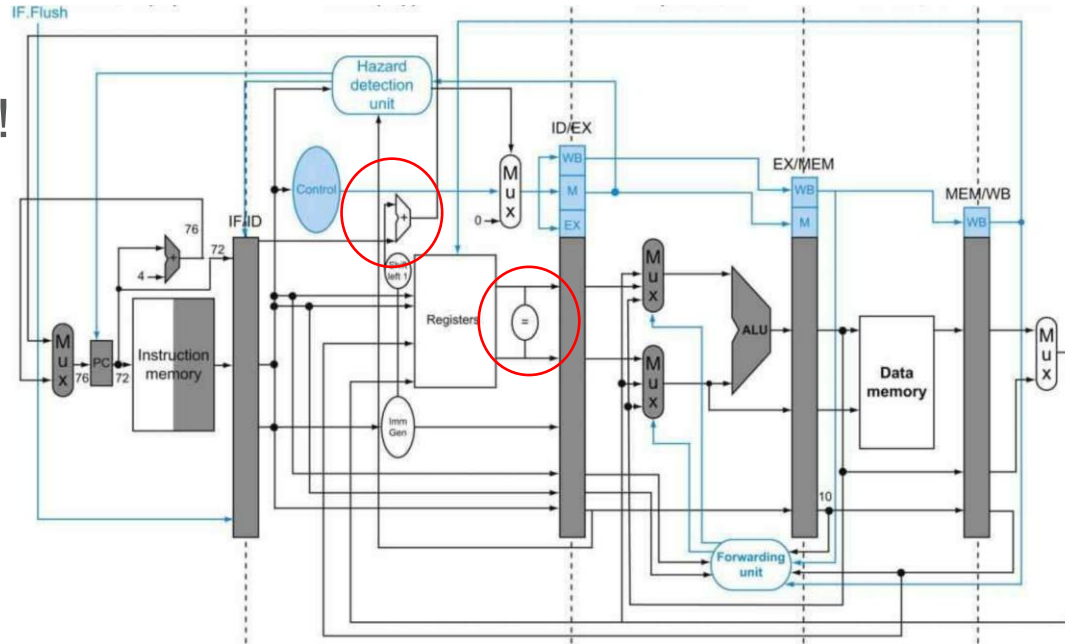
Control Hazards: Speculation

- Reducing Misprediction Cost
 - Resolving branches sooner
 - Making it 2 cycles can be done



Control Hazards: Speculation

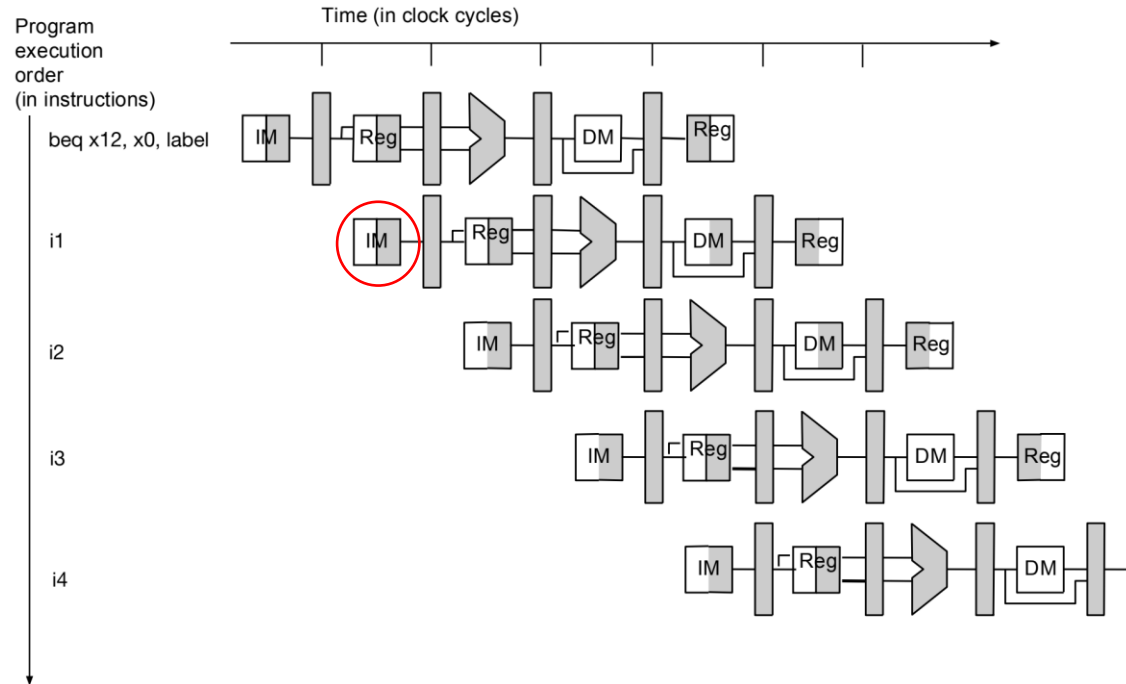
- Reducing Misprediction Cost: More aggressive
 - Move branch cond to ID:
 - Move branch target to ID
- Warning: r1/r2 RAW
 - Need data forwarding!



Whiteboard: Aggressive branch opt

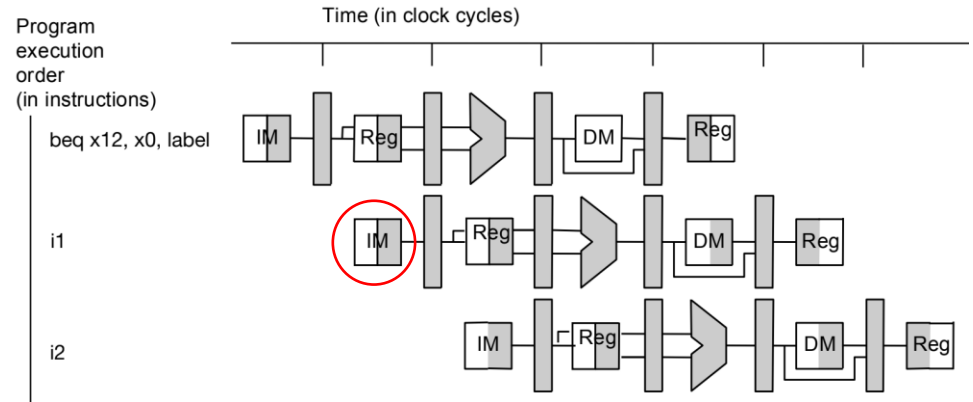
Control Hazards: Speculation

- Predict: Always Taken
- What do we need?
 - Condition
 - Target



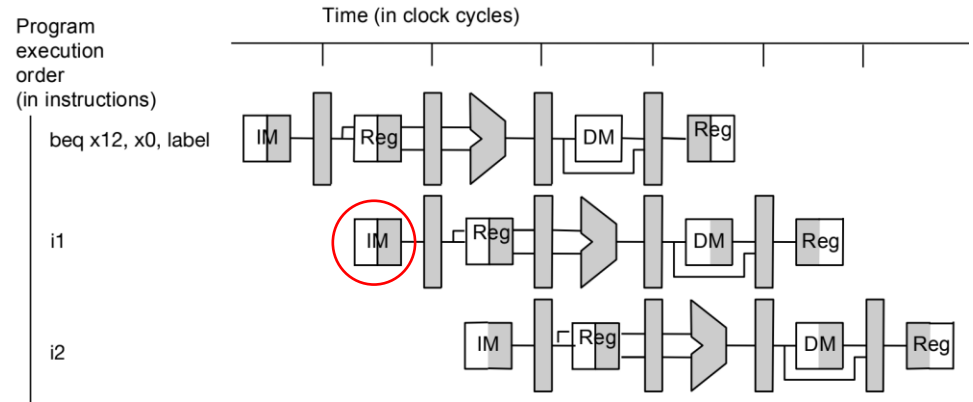
Control Hazards: Speculation

- Predict: Always Taken
 - How to obtain NextPC?
 - Needed in IF stage
 - We can predict



Control Hazards: Speculation

- Predict: Always Taken
 - How to obtain the branch target?
 - We can predict



Control Hazards: Speculation

- When do we know the branch target?
 - Direct Jumps, Function calls: JAL
 - $PC = PC + imm$
 - In ID stage
 - Prediction accuracy: High

Control Hazards: Speculation

- When do we know the branch target?
 - Hard to compute conditional branches: BEQ/BNE
 - $PC = (R[rs1] \text{ ops } R[rs2]) ? (PC + imm) : PC + 4$
 - In ID stage
 - Prediction accuracy: High

Control Hazards: Speculation

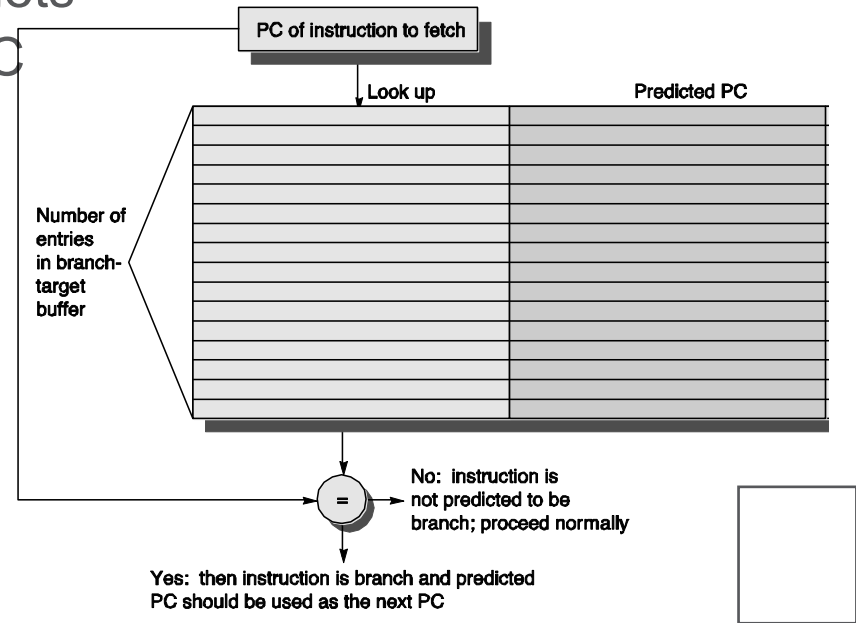
- When do we know the branch target?
 - Indirect Jumps, Dynamic Calls, Returns: JALR
 - $PC = R[rs1] + imm$
 - After ID stage
 - Prediction accuracy: Low

Control Hazards: Speculation

- How do we predict the address?
 - We need to track the history of previous branch targets
 - If a branch was visited before, reuse its previous target
 - There is a chance it will execute the same path

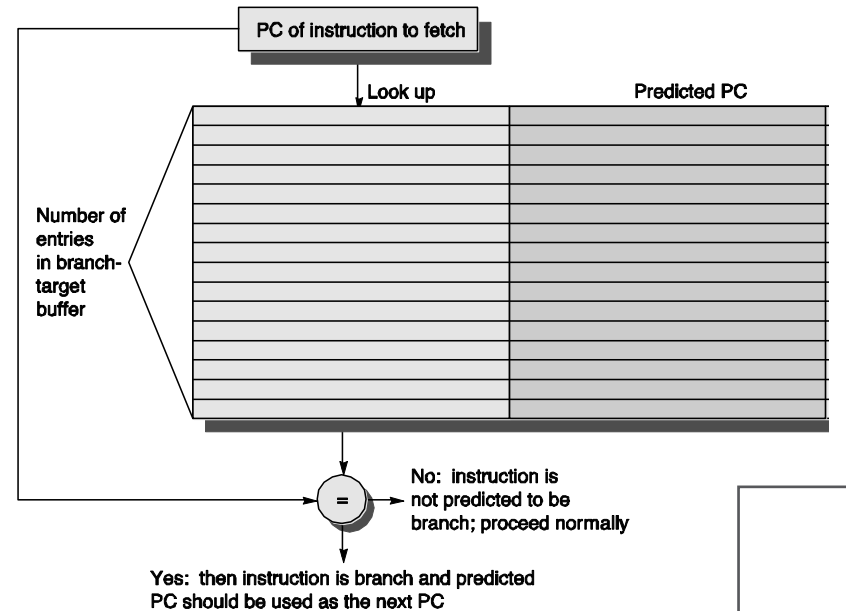
Control Hazards: Speculation

- Branch Target Buffer (BTB)
 - A table that stores branch targets
 - Indexable using the current PC
 - Why using the PC?



Control Hazards: Speculation

- How BTB works?
 - Use PC to lookup the BTB
 - Found a match?
 - NextPC = BTB entry;
 - Otherwise
 - NextPC = PC + 4;



Control Hazards: Speculation

- BTB implementation questions:
 - Where is BTB accessed?
 - How do we know that the current PC is a branch before accessing the BTB?
 - When do we update the BTB?
 - How do we update the BTB?

Control Hazards: Speculation

- Where is BTB accessed?
 - NextPC is needed in IF stage
 - BTB should be accessed in IF stage

Control Hazards: Speculation

- How do we know that the current PC is a branch before accessing the BTB?
 - Only branches are added into BTB
 - Make sure to clear BTB at startup

Control Hazards: Speculation

- When do we update the BTB?
 - When we can compute branch target
 - In ID stage:
 - Add dedicated adder
 - JALR timing issue – register file access
 - In EX stage:
 - Add dedicated adder
 - No timing issue

Control Hazards: Speculation

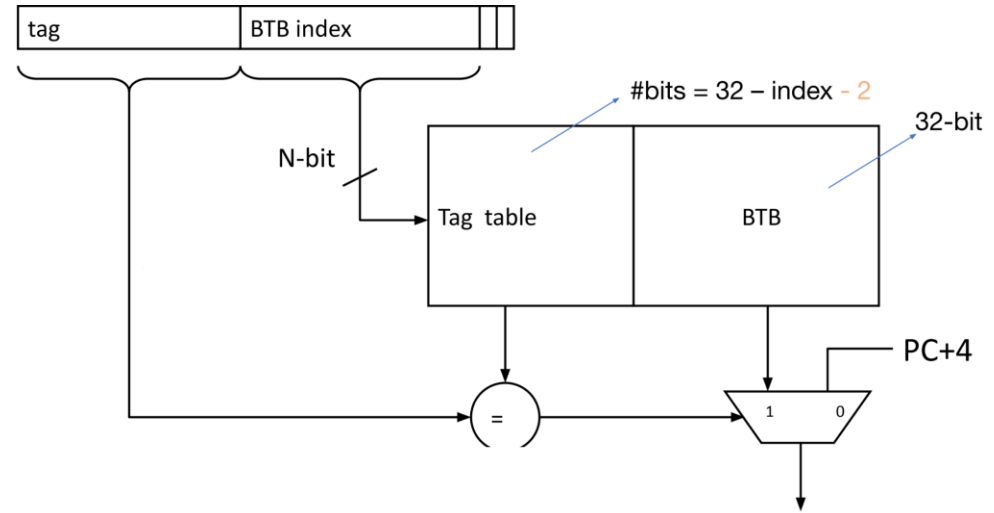
- How do we update the BTB?
 - Check if branch
 - Calculate NextPC
 - Update BTB
 - $BTB[PC] = NextPC$

Control Hazards: Speculation

- BTB Hardware Implementation
 - Implementing a hash table in H/W is expensive
 - Lookup area cost
 - Lookup latency
 - Table size must be small

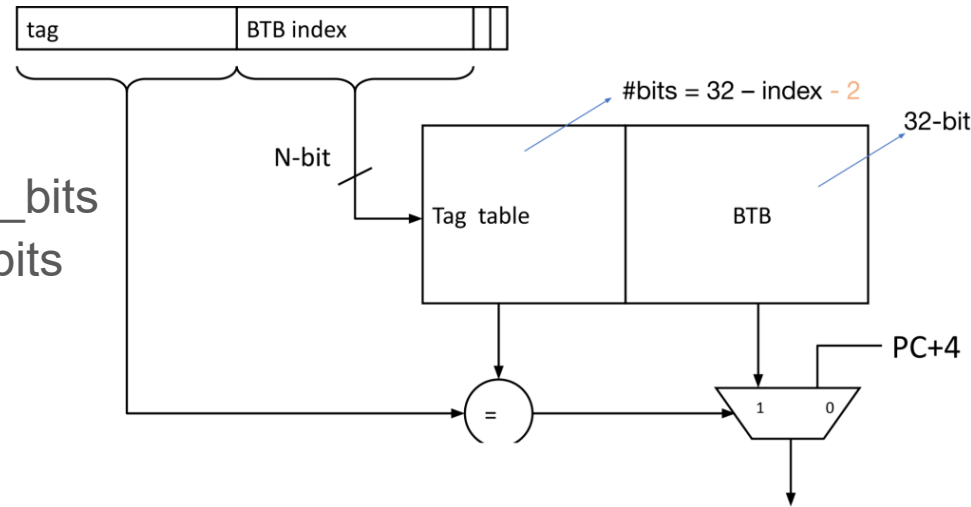
Control Hazards: Speculation

- BTB Hardware Implementation
 - Use a Tagged-based Lookup table
 - Fixed size: `btb_size`
 - Indexing: `btb_index`
 - Lookup: `btb_tag`



Control Hazards: Speculation

- BTB Hardware Implementation
 - PC address is 4-byte aligned
 - lower 2 bits are zero
 - Formulas:
 - $\text{index_bits} = \log_2(\text{btb_size})$
 - $\text{pc_index} = (\text{pc} \gg 2) \& \text{index_bits}$
 - $\text{pc_tag} = (\text{pc} \gg 2) \gg \text{index_bits}$



Control Hazards: Speculation

- BTB Hardware Implementation

- Initialization:
 - Clear valid bit at reset
- BTB lookup:
 - `btb <= BTB[pc_index];`
 - `if (btb.valid == 1 && btb.tag == pc_tag) { PC = btb.target; } else { PC = PC + 4; }`
- BTB update:
 - `BTB[pc_index] <= {1, pc_tag, target};`

BTB hardware table

valid 1 bit	tag 26 bits	target 30 bits	16 entries ↓

Control Hazards: Speculation

- Improving Branch prediction
 - Predicting always taken is about 70% accurate
 - Better than always not taken (30%).
 - How do we improve the prediction accuracy?

Control Hazards: Speculation

- Code reordering optimization
 - If H/W implements static prediction
 - Not Taken or always Taken
 - Reorder code to prioritize predicted path
 - Pros/cons?
 - Pros: improved accuracy
 - Cons: architecture specific

Control Hazards: Speculation

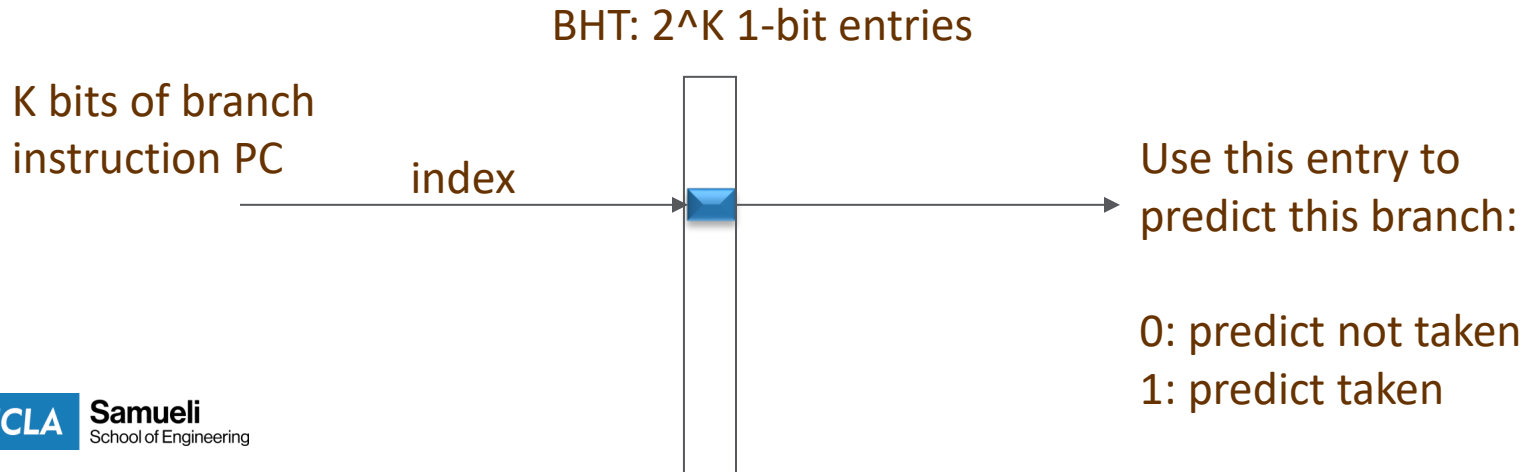
- BTFNT (Backward Taken, Forward Not Taken)
 - Loops usually have a few iterations, so this is like always predicting that the loop is taken
 - Loop back-edge => backward direction
 - Pros/Cons?
 - Pros: Improved accuracy
 - Cons: limited to loops only

Control Hazards: Speculation

- Using a Branch predictor
 - Predict the outcome dynamically using the branch history
 - Recurring branches tend to repeat their pattern (e.g. loops)

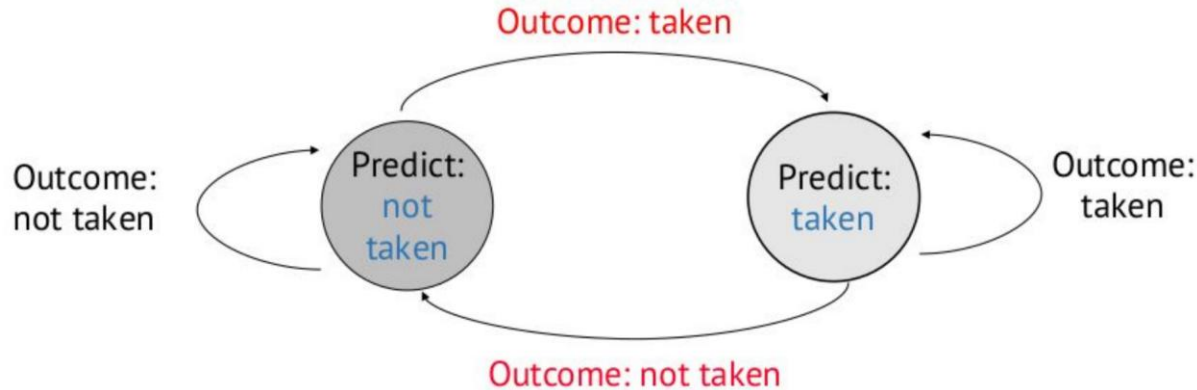
Control Hazards: Speculation

- One-Bit Branch Predictor
 - BTB entry hold a single bit: taken=1 or not_taken=0.
 - Index the table using lower K bits of PC
 - Update the table when the condition is resolved.



Control Hazards: Speculation

- One-Bit Branch Predictor
 - FSM



Control Hazards: Speculation

- One-Bit Branch Predictor
 - Examples

DC08: TTTTTTTTTT ... TTTTTTTTTNTTTTTTTTTT...

← 100,000 iterations →

How often is branch outcome != previous outcome?

2 / 100,000

99.9%

DC44: TTTTTT ... TNTTTTTT ... TNTTTTTT ...

← 2 / 100 →

2 / 100

98.0%

DC50: TNTNTNTNTNTNTNTNTNTNTNTNTNTNTNT...

← 2 / 2 →

2 / 2

00.0%

Control Hazards: Speculation

- One-Bit Is not enough!

Example: short loop (8 iterations)

- Taken 7 times, then not taken once
- Act: TTTTTTTNTTTTTTNTTTTTTTNT...
- Pred: XTTTTTTT**T**NTTTTTT**T**NTTTTTTT**T**N
- Corr: Xoooooooo**MM**oooooooo**MM**oooooooo**MM**
- Misprediction rate: $2/8 = 25\%$

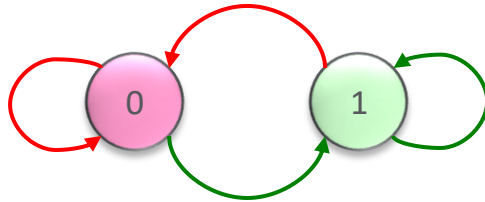
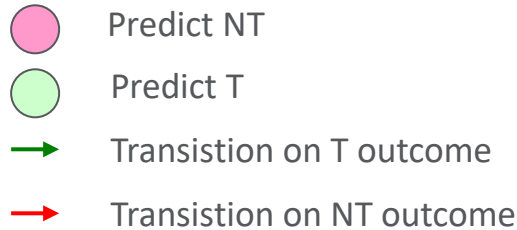
Execute the same loop again

- First always mispredicted (previous outcome was not taken)
- Then 6 predicted correctly
- Then last one mispredicted again

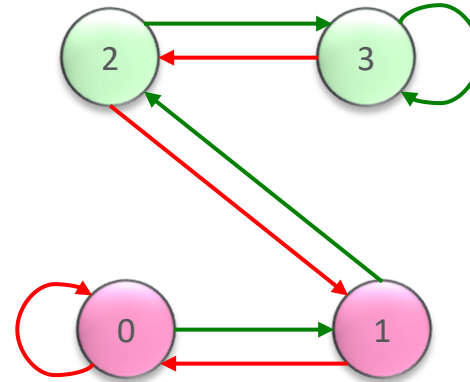
Each fluke/anomaly results in two mispredicts per loop

Control Hazards: Speculation

- Two-Bit Branch Predictor



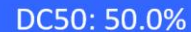
FSM for Last-time
Prediction



FSM for 2bC
(2-bit Counter)

Smith, ISCA 1981

- Two-Bit Branch Predictor



Control Hazards: Speculation

- Can we do better?
 - 1-BC and 2-BC predictors use last branch outcome to predict
 - Observation1: global branch correlation
 - The branch outcome is correlated with outcome from previous branches
 - Observation2: local branch correlation
 - The branch outcome is correlated with prior-outcome from same branch

Control Hazards: Speculation

Global branch correlation

```
if (cond1) {  
    --/--  
}  
if (cond1 and cond2) {  
    --/--  
}
```

Local branch correlation

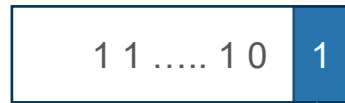
```
for (x: 0 -> 1023) {  
    --/--  
}
```

Control Hazards: Speculation

Two-level Branch Predictor

- Level1: history of last N branches
- Level2: last direction the branch took

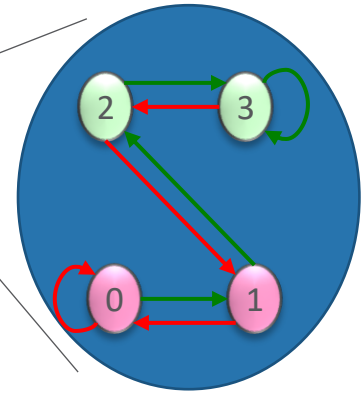
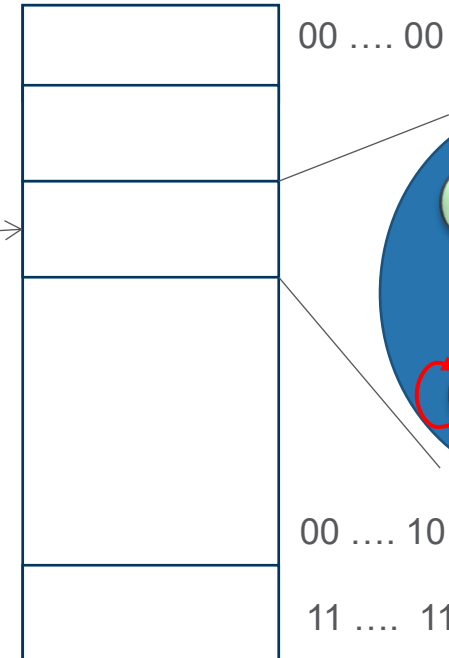
Branch History Register (BHR)



previous one

index

2-Bit BHT



Yeh&patt'92

Control Hazards: Speculation

Two-level Branch Predictor

Initialization value (0 or 1)

Past

Present



← History length →

1 : branch is taken
0: branch is not-taken

New BHR = old BHR<<1 | (br_dir);

Example:

	BHR 00000
Br1 : taken	→ BHR 00001
Br2: not-taken	→ BHR 00010
Br3: taken	→ BHR 00101

Control Hazards: Speculation

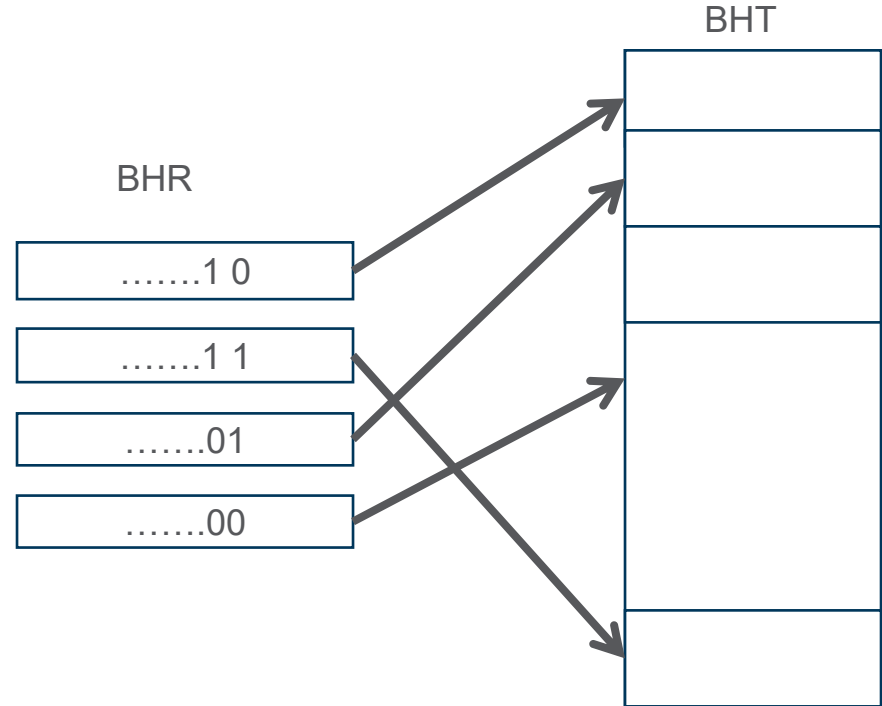
Two-level Branch Predictor

Branch X: if (cond1)

Branch Y: if (cond 2)

Branch Z : if (cond 1 and cond 2)

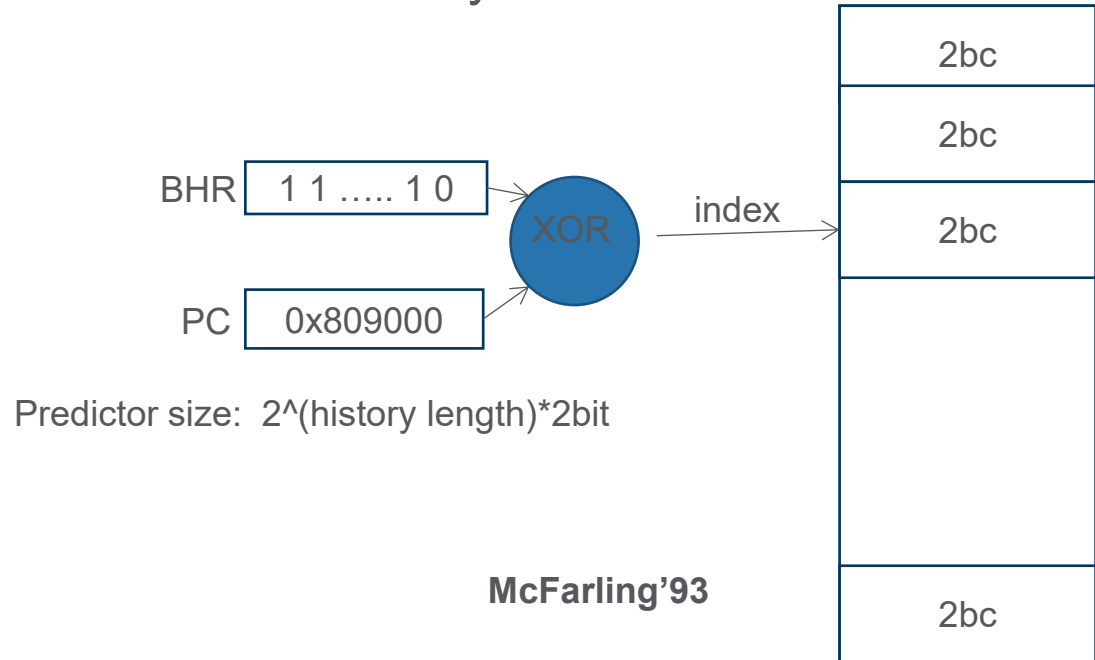
Branch X	Branch Y	Branch Z
1	0	0
1	1	1
0	1	0
0	0	0



Control Hazards: Speculation

Gshare Branch Predictor

- Different branches may see the same history but have different outcome
- Need additional context



McFarling'93

Control Hazards: Speculation

Gshare Branch Predictor: algorithm

```
predict_func(pc, actual_dir) {  
    index = PC xor BHR  
    taken = (2bit_counters[index] >= 2) ? 1 : 0  
    correctly_predicted = (actual_dir == taken) ? 1 : 0 // stats  
}
```

```
update_func(pc, actual_dir) {  
    index = PC xor BHR  
    if (actual_dir)  
        INC_SAT(2bit_counter[index])  
    else  
        DEC_SAT(2bit_counter[index])  
    BHR = BHR << 1 | actual_dir  
}
```

Control Hazards: Speculation

Improving Return Call Prediction

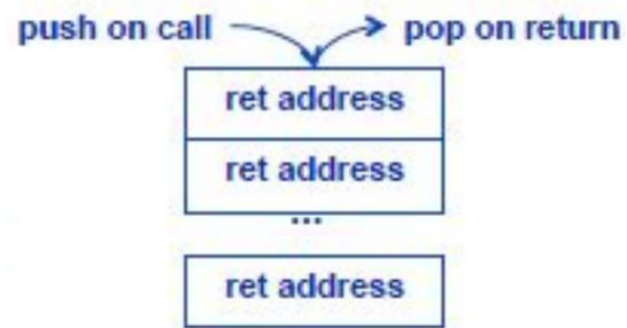
- RISC-V JALR
 - $PC = R[rs1] + imm$
- Target depends on rs1

Control-Flow inst.	Direction at fetch time	Number of possible next fetch addresses?	When is next fetch address resolved?
Conditional	Unknown	2	Decode ($rs1 == rs2$)
Unconditional	Always taken	1	Decode (PC + offset)
Call	Always taken	1	Decode (PC + offset)
Return	Always taken	Many	Decode (register dependent)
Indirect	Always taken	Many	Decode (register dependent)

Control Hazards: Speculation

Return Stack Buffer (RSB)

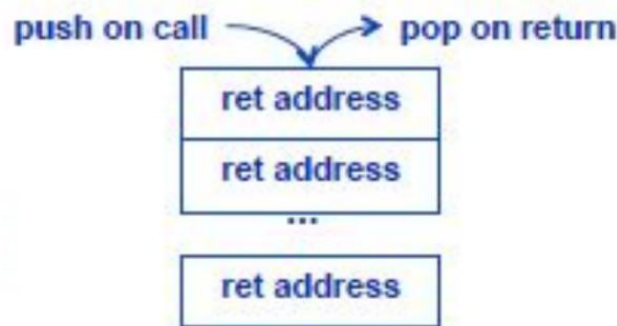
- Use hardware stack
 - About 4-8 entries
- On function call:
 - Push current return address
- On function return:
 - Pop last return address
 - Use that for prediction



Control Hazards: Speculation

Return Stack Buffer (RSB)

- Tail recursion optimization
 - if (stack.top == ra) => skip
- Integration with BTB:
 - Predict NextPC from BTB or RSB
 - How to choose?
 - Extend BTB with “isRA” column



Acknowledgement

- This course is partly inspired and use materials from the following courses created by my colleagues:
 - ECE-M1126C: Nader Sehatbakhsh @ UCLA
 - CS3220: Hyesoon Kim @ gatech

Q&A
